

Post-Quantum Adaptor Signatures with Strong Security from Cryptographic Group Actions

Ryann Cartor¹, Nathan Daly², Giulia Gaggero³, Jason T. LeGrow^{2,4}, Andrea Sanguineti⁵, and Silvia Sconza⁶

¹ School of Mathematical and Statistical Sciences, Clemson University, Clemson, South Carolina, USA

² Department of Mathematics, Virginia Tech, Blacksburg, Virginia, USA

³ Department of Mathematics, McMaster University, Hamilton, Ontario, Canada

⁴ Virginia Tech Center for Quantum Information Science and Engineering

⁵ Dipartimento di Matematica, Università di Genova, Genova, Italy

⁶ Institute of Mathematics, University of Zurich, Zurich, Switzerland

Abstract. We present One Round “Cheating” Adaptor Signatures (OR-CAS): a novel and efficient construction of adaptor signature schemes from CSI-FiSh. Our protocol improves substantially on existing group action-based schemes: Unlike IAS (Tairi et al., FC 2021), our scheme does not require expensive non-interactive zero-knowledge proofs, and unlike adaptor MCSI-FiSh (Jana et al., CANS 2024) our construction does not require any modification to the underlying digital signature scheme. We prove the protocol’s security under the strong security notions of Dai et al. (Indocrypt 2022) and Gerhart et al. (Eurocrypt 2024).

Keywords: Digital Signatures · Adaptor Signatures · Post-Quantum Cryptography · Group Action-Based Cryptography

1 Introduction

Adaptor signatures—first introduced by Poelstra [23]—augment a digital signature scheme by incorporating an additional “pre-signing” phase, in which a user produces a pre-signature that can be transformed into a proper signature once a witness y to a statement Y of an NP language is revealed. The prototypical use case is the *atomic swap*, which allows two mutually untrusting parties to securely exchange digital assets, without using a trusted party. Aumayr *et al.* [3] formalize security notions that guarantee adaptor signatures’ suitability for atomic swaps: adaptive existential unforgeability, pre-signature adaptability, and witness extractability.

Beyond this basic functionality, adaptor signature schemes have been used for many constructions for blockchain, such as payment channels [22] and generalized channels [3], deterministic wallets [13, 14], private coin mixing protocols [26, 17], oracle-based payments [21], and multiparty fair exchange [4]. Recent works due to Dai *et al.* [12] and Gerhart *et al.* [15, 16] demonstrate that Aumayr *et al.*’s security definitions do not guarantee security in these applications. These

works define a number of stronger security notions—full extractability, unique extractability, unlinkability, and pre-verify soundness—to suit these new protocols. Gerhart *et al.* also present a number of protocol constructions that satisfy these new definitions. The most notable among these are *dichotomic signatures*, which generalize Schnorr signatures. Gerhart *et al.* demonstrate that a number of existing digital signature schemes can be framed as dichotomic signatures, and thus transformed into secure adaptor signatures.

Novel security notions notwithstanding, the vast majority of existing adaptor signature schemes—including all schemes constructed and analyzed in [15, 16]—face a serious threat: quantum cryptanalysis. Indeed, any scheme based on the difficulty of discrete logarithms or factoring is broken by Shor’s algorithm, and so will not be secure once large-scale quantum computers are built. To ensure that these functionalities can be realized securely in the future, we develop *post-quantum* protocols, which run on classical computers but resist attacks by quantum adversaries. *Cryptographic group actions* [2] are a promising technique that underlie many many post-quantum digital signatures, including CSI-FiSh [8], LESS [9], and MEDS [10]. One major benefit of schemes built in this way is that they can often be easily modified to provide advanced functionalities, including ring signatures [7], threshold signatures [5], and blind signatures [19, 20]. Despite this, there are only two group action-based adaptor signature constructions in the literature: Isogeny Adaptor Signatures (IAS) [27] and adaptor modified CSI-FiSh [18]. These constructions have many disadvantages: (1) they have not been proven secure in the security model of Gerhart *et al.*; (2) IAS requires a large, expensive non-interactive zero-knowledge proof, making pre-signatures between 10 and 70 times as large as signatures, and taking between 15 and 20 times as long to produce as signatures; and (3) adaptor modified CSI-FiSh is not directly compatible with CSI-FiSh, and is instead built on a non-standard digital signature, limiting its usefulness to only those blockchains which choose to support the modified signature scheme. These drawbacks make existing group action-based schemes unsuitable for practical applications, and naturally lead us to the following question:

Is it possible to construct efficient group action-based adaptor signatures without modification to the underlying signature scheme?

1.1 Our Contributions

We answer this question in the affirmative with our new construction, ORCAS. More precisely, ORCAS improves upon existing group action-based adaptor signatures in several ways:

Standard underlying signature: In contrast with MCSI-FiSh [18], our scheme does not require any modification to the underlying digital signature scheme.

It is compatible with CSI-FiSh, as specified in [8].

Security: Our scheme is the only post-quantum adaptor signature scheme that is compatible with an unmodified underlying signature which provably sat-

isfy the strong security notions of Dai *et al.* [12]. It is also the only post-quantum scheme that is proven to satisfy the pre-verify soundness property introduced by Gerhart *et al.* [16].

Efficiency: We exploit the small challenge space of group action-based signatures to avoid IAS’ expensive proof. This results in pre-signatures only a byte longer than the signatures themselves. Our pre-signatures are 8 to 36 times shorter than IAS pre-signatures at the same parameter sizes.

Our scheme is built from a sigma protocol using the Fiat-Shamir heuristic, like the Schnorr scheme [25]. While the Schnorr protocol has an exponential challenge space and attains strong soundness in a single round, CSI-FiSh has a constant-size challenge space, and must amplify its soundness through repetition. Generally this is an undesirable feature, since it leads to larger signatures and slower signing and verifying. But our construction actually exploits this limitation in an essential way, yielding a Schnorr-inspired adaptor signature protocol with no Schnorr-based analogue. Our scheme is built on three observations:

1. As in both MCSI-FiSh [18] and the optimized version of IAS [27], it is only necessary to adapt a single round of the signature.
2. The expensive NIZK proof in IAS [27] is only necessary if the challenge is nonzero; as in MCSI-FiSh [18], it is easy to adapt a round in which the challenge is guaranteed to be 0.
3. The extra round in MCSI-FiSh [18] is not necessary in order to have a round whose challenge is predictably 0; for a constant-sized challenge space, any sequence of randomly chosen rounds has sufficiently high probability of including at least one round with challenge 0.

Of course, we aim to prove that ORCAS is a secure adaptor signature scheme for CSI-FiSh. There are a number of small differences between them that make the proof difficult. To make the proof easier, we introduce both a modified version of CSI-FiSh and a highly unoptimized version of ORCAS, for which the security proofs are relatively straightforward. We then prove a general set of lemmas that establish that, at a high level, if AS is a adaptor signature that is secure for the signature scheme Σ , then for any adaptor signature AS' and signature Σ' that are “sufficiently similar” to AS and Σ , respectively, AS' will be a secure adaptor signature for Σ' . With this very general result established, we show that both versions of ORCAS are sufficiently similar, as are both versions of CSI-FiSh. We present these ideas formally in Sections 4 and 5. We expect that our techniques will see applications to other adaptor signature constructions.

Aside from our new protocol and proof techniques, we also include some new analyses of the definitions of [16], which demonstrate the need for novel adaptor signature constructions in the post-quantum setting.

2 Background and Definitions

For brevity, we defer basic definitions related to hard relations, non-interactive zero-knowledge proofs, and ordinary digital signatures to Appendix A.

2.1 Adaptor Signatures

An adaptor signature scheme extends a digital signature scheme by incorporating a “pre-signing” phase, in which parties irrevocably commit to signing messages when specified conditions are met. In particular, Alice and Bob can agree on messages m_A, m_B , and construct pre-signatures (intuitively, commitments to signatures) on them. Alice will have the ability to transform (called “adapting” in this context) Bob’s pre-signature into a signature on m_B , but by doing so she will reveal enough information for Bob to adapt Alice’s pre-signature into a signature on m_A . In this way, each party is assured that if their message becomes signed, they can produce a signature on their partner’s message.

Adaptor signatures are formally defined in Definition 2.1.

Definition 2.1 (Adaptor Signature Scheme [15]). *An adaptor signature scheme with respect to a digital signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ and a relation $\mathcal{R}$ with underlying language $\mathcal{L}$ is a tuple $\text{AS} = (\text{PreSign}, \text{PreVerify}, \text{Adapt}, \text{Ext})$ of algorithms defined as follows:*

PreSign(sk, m, Y): *On input a secret key sk , a message m , and a statement $Y \in \mathcal{L}$, the pre-signing algorithm outputs a pre-signature $\hat{\sigma}$.*

PreVerify($\text{pk}, m, Y, \hat{\sigma}$): *On input a public key pk , a message m , a statement $Y \in \mathcal{L}$, and a pre-signature $\hat{\sigma}$, the pre-verification algorithm outputs a bit $b \in \{0, 1\}$.*

Adapt($\text{pk}, \hat{\sigma}, y$): *On input a public key pk , a pre-signature $\hat{\sigma}$, and a witness y for the relation $\mathcal{R}$, the adapting algorithm outputs a signature σ .*

Ext($\sigma, \hat{\sigma}, Y$): *On input a signature σ , a pre-signature $\hat{\sigma}$, and a statement $Y \in \mathcal{L}$, the extraction algorithm outputs either a witness y such that $(Y, y) \in \mathcal{R}$, or $\perp$.*

Alice and Bob can use an adaptor signature scheme to exchange digital goods securely across blockchains, as follows: first, Alice and Bob agree to exchange Alice’s good A for Bob’s good B. Let Alice’s message m_A encode “Send good A to Bob”, and Bob’s message m_B encode “Send good B to Alice”. Then

1. Alice pre-signs m_A and publishes the pre-signature.
2. Bob pre-signs m_B and publishes the pre-signature.
3. Alice adapts Bob’s pre-signature into a signature, and records this signature on the blockchain. This authorizes sending Bob’s good B to Alice.
4. Bob uses his pre-signature and signature (obtained from the blockchain) to adapt Alice’s pre-signature into a signature, which he records on the blockchain. This authorizes sending Alice’s good A to Bob.

The following correctness notion intuitively requires that if the scheme is used honestly, then all adaptor signature functionalities work as expected.

Definition 2.2 (Pre-Signature Correctness [3, Definition 1]). *We say that AS is pre-signature correct if for all $\lambda \in \mathbb{N}$, all $(Y, y) \in \mathcal{R}$, and any message*

m , we have

$$\mathbb{P} \left[\begin{array}{l} \text{PreVerify}(\text{pk}, m, Y, \hat{\sigma}) = 1 \\ \text{Verify}(\text{pk}, m, \sigma) = 1 \\ (Y, y') \in \mathcal{R} \end{array} \middle| \begin{array}{l} (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda) \\ \hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y) \\ \sigma \leftarrow \text{Adapt}(\hat{\sigma}, y) \\ y' \leftarrow \text{Ext}(\hat{\sigma}, \sigma, Y) \end{array} \right] = 1.$$

Security Notions. Recent works due to Dai *et al.* [12] and Gerhart *et al.* [15, 16] identify gaps in existing adaptor signature security definitions [3], provide examples of schemes that satisfy older definitions but which still fail to satisfy security in natural real-world applications, and propose new security definitions that guarantee security in those applications. We adopt the definitions of [12, 15, 16] in our work, and state them here for completeness.

Pre-Signature Adaptability. This property guarantees that any valid pre-signature with respect to a statement in the language of the hard relation can be adapted into a valid signature. This prevents a cheating user from providing a pre-signature that pre-verifies but cannot be adapted, which would break the security of the atomic swap protocol. Formally, we have the following definition.

Definition 2.3 (Pre-Signature Adaptability.) *An adaptor signature scheme AS satisfies pre-signature adaptability if for all $\lambda \in \mathbb{N}$, $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$, $(Y, y) \in \mathcal{R}$, and $m \in \{0, 1\}^*$, if $\hat{\sigma}$ satisfies $\text{PreVerify}(\text{pk}, m, Y, \hat{\sigma}) = 1$, then $\text{Verify}(\text{pk}, m, \text{Adapt}(\hat{\sigma}, y)) = 1$.*

Unlinkability and Canonical Signing. Unlinkability guarantees that ordinary signatures (produced using the signing algorithm of the underlying signature scheme) cannot be distinguished from adapted pre-signatures. Thus, a third-party observer who sees the signatures resulting from an atomic swap cannot identify those signatures as having arisen from an atomic swap.

Definition 2.4 (Unlinkability [12, Section 3]). *Let Unlinkability be as defined in Figure 1. For an adversary $\mathcal{A}$, define the adversary's advantage as $\text{Adv}(\mathcal{A}) = 2\mathbb{P}[\text{Unlinkability}_{\mathcal{A}, \text{AS}}(\lambda) = 1] - 1$. We say that an adaptor signature scheme AS is unlinkable if for all PPT adversaries $\mathcal{A}$, there exists a negligible function ν such that for all $\lambda \in \mathbb{N}$, $\text{Adv}(\mathcal{A}) \leq \nu(\lambda)$.*

A related notion is *canonical signing*, which requires that for any message m and secret key sk , ordinary signatures are distributed identically to signatures coming from the *canonical signing algorithm* CanonicalSign , defined in Figure 2. We also define *strongly canonical signing*: the output of Sign is distributed identically to that of $\text{CanonicalSign}_{(Y, y)}$ (Figure 2), for *every* statement-witness pair $(Y, y) \in \mathcal{R}$. It is not hard to see that strongly canonical signing implies canonical signing.

In addition to unlinkability, we define a new security property called *weak unlinkability*, which guarantees that, even if an observer may be able to distinguish an adapted pre-signature from a standard signature, it is still hard to link

Unlinkability $_{\mathcal{A}, \text{AS}}(\lambda)$	Challenge($b, m, (Y, y)$)
101 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$	201 : assert $(Y, y) \in R$
102 : $b \leftarrow \$ \{0, 1\}$	202 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
103 : $b' \leftarrow \mathcal{A}^{\mathcal{O}_S, \mathcal{O}_{\text{PS}}(\cdot, \cdot), \text{Challenge}(b, \cdot, \cdot)}(\text{pk})$	203 : $\sigma_0 \leftarrow \text{Adapt}(\hat{\sigma}, y)$
104 : return $[b = b']$	204 : $\sigma_1 \leftarrow \text{Sign}(\text{sk}, m)$
	205 : return σ_b
$\mathcal{O}_S(m)$	$\mathcal{O}_{\text{PS}}(m, Y)$
301 : $\sigma \leftarrow \text{Sign}(\text{sk}, m)$	401 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
302 : return σ	402 : return $\hat{\sigma}$

Fig. 1: The unlinkability game.

CanonicalSign(sk, m)	CanonicalSign $_{(Y, y)}(\text{sk}, m)$
101 : $(Y, y) \leftarrow \text{Rgen}(1^\lambda)$	201 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
102 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$	202 : $\sigma \leftarrow \text{Adapt}(\hat{\sigma}, y)$
103 : $\sigma \leftarrow \text{Adapt}(\hat{\sigma}, y)$	203 : return σ
104 : return σ	

Fig. 2: The canonical signing algorithm for an adaptor signature scheme AS.

an adapted pre-signature to the statement with respect to which it was generated. In practice, this means that even a user publishing a signature cannot hide the fact that they participated in an atomic swap, any information beyond that remains private. This privacy guarantee should be sufficient for most applications.

Definition 2.5 (Weak Unlinkability). *An adaptor signature scheme AS for the signature/relation pair $(\Sigma, \mathcal{R})$ is weakly unlinkable if for any PPT $\mathcal{A}$,*

$$\mathbb{P}[\text{WeakUnlinkability}^{\mathcal{A}}(1^\lambda) = 1] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where WeakUnlinkability is as defined in Figure 3.

It is straightforward to verify that unlinkability always implies weak unlinkability, and the converse holds if AS has canonical signing for Σ .

(Strong) Full Extractability and Extractability. Dai *et al.*'s notion of (strong) full extractability replaces the earlier notions of witness extractability [3, Definition 4] and existential unforgeability [3, Definition 2], which are now known to be insufficient for some applications. This new notion guarantees that it is infeasible for any adversary to produce a valid signature (on a new message for full extractability, on any message for strong full extractability) except by adapting a pre-signature given with respect to a statement that has a known witness.

WeakUnlinkability(1^λ)	Challenge($m, (Y_0, y_0)$)
101 : $(\text{sk}, \text{pk}) \leftarrow \text{KGen}(1^\lambda)$	201 : assert $(Y_0, y_0) \in \mathcal{R}$
102 : $b \leftarrow \$ \{0, 1\}$	202 : $(Y_1, y_1) \leftarrow \text{Rgen}()$
103 : $b' \leftarrow \$ \mathcal{A}^{\mathcal{O}_S, \mathcal{O}_{PS}, \text{Challenge}}(\text{pk})$	203 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y_b)$
104 : return $[b = b']$	204 : $\sigma \leftarrow \text{Adapt}(\hat{\sigma}, y_b)$
	205 : return σ
$\mathcal{O}_S(m)$	$\mathcal{O}_{PS}(m, Y)$
301 : $\sigma \leftarrow \text{Sign}(\text{sk}, m)$	401 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
302 : return σ	402 : return $\hat{\sigma}$

Fig. 3: The weak unlinkability game.

Definition 2.6 (Full Extractability [12, Figure 3]). An adaptor signature scheme AS satisfies full extractability if, for every PPT adversary $\mathcal{A}$ there exists a negligible function ν such that for all $\lambda \in \mathbb{N}$, $\mathbb{P}[\text{FullExtractability}_{\mathcal{A}, \text{AS}}(\lambda) = 1] \leq \nu(\lambda)$, where FullExtractability is defined in Figure 4.

Definition 2.7 (Strong Full Extractability [12, Figure 3]). An adaptor signature scheme AS satisfies strong full extractability if, for every PPT adversary $\mathcal{A}$ there exists a negligible function ν such that for all $\lambda \in \mathbb{N}$, $\mathbb{P}[\text{StrongFullExtractability}_{\mathcal{A}, \text{AS}}(\lambda) = 1] \leq \nu(\lambda)$, where StrongFullExtractability is defined in Figure 4.

FullExtractability(λ)	StrongFullExtractability(λ)
101 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$	
102 : $C, T, S, U \leftarrow \emptyset$	
103 : $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{NewY}}, \mathcal{O}_S, \mathcal{O}_{\text{PS}}(\cdot, \cdot)}(\text{pk})$	
104 : assert $\text{Verify}(\text{pk}, m^*, \sigma^*) = 1$	
105 : assert $m^* \notin S$	
106 : assert $(m^*, \sigma^*) \notin U$	
107 : return $((Y, \text{Ext}(Y, \hat{\sigma}, \sigma^*)) \notin \mathcal{R} \ \forall (Y, \hat{\sigma}) \in T[m^*] \text{ such that } Y \notin C)$	
$\mathcal{O}_S(m)$	$\mathcal{O}_{\text{PS}}(m, Y)$
201 : $\sigma \leftarrow \text{Sign}(\text{sk}, m)$	301 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
202 : $S \leftarrow S \cup \{m\}$	302 : $T[m] \leftarrow T[m] \cup \{(Y, \hat{\sigma})\}$
203 : $U \leftarrow U \cup \{(m, \sigma)\}$	303 : return $\hat{\sigma}$
204 : return σ	

Fig. 4: The FullExtractability and StrongFullExtractability games. Lines in gray appear only in the FullExtractability game.

The authors also introduce a simpler security property, *extractability*, which is equivalent to full extractability if $\mathcal{R}$ is a hard relation and the scheme has canonical signing.

Definition 2.8 (Extractability [12, Figure 5]). *An adaptor signature scheme AS is extractable if, for every PPT adversary $\mathcal{A}$ there exists a negligible function ν such that for all $\lambda \in \mathbb{N}$, $\mathbb{P}[\text{Extractability}_{\mathcal{A}, \text{AS}}(\lambda) = 1] \leq \nu(\lambda)$, where Extractability is the game defined in Figure 5.*

$\text{Extractability}_{\mathcal{A}, \text{AS}}(\lambda)$	$\mathcal{O}_{\text{PS}}(m, Y)$
101 : $T \leftarrow \emptyset$	201 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
102 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$	202 : $T[m] \leftarrow T[m] \cup \{(Y, \hat{\sigma})\}$
103 : $(m, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{PS}}(\cdot, \cdot)}(\text{pk})$	203 : return $\hat{\sigma}$
104 : assert $\text{Verify}(\text{pk}, m, \sigma) = 1$	
105 : for $(Y, \hat{\sigma}) \in T[m]$	
106 : assert $(Y, \text{Ext}(\sigma, \hat{\sigma}, Y)) \notin \mathcal{R}$	
107 : return 1	

Fig. 5: The extractability game.

Unique Extractability. Unique extractability guarantees that it is infeasible for any efficient adversary to generate a pre-signature for some message and statement that corresponds to two distinct valid signatures on the message. As demonstrated in [16, Section 4.1], some adaptor signature applications can be rendered insecure if this property is not satisfied—e.g., the coin-mixing protocol of [17].

Definition 2.9 (Unique Extractability [16, Definition 13]). *An adaptor signature scheme AS is uniquely extractable if for every PPT adversary $\mathcal{A}$ there exists a negligible function ν such that for every $\lambda \in \mathbb{N}$, $\mathbb{P}[\text{UniqueExtractability}_{\mathcal{A}, \text{AS}}(\lambda) = 1] \leq \nu(\lambda)$, where UniqueExtractability is the game defined in Figure 6.*

Pre-verify soundness. While pre-signature adaptability ensures that, for any honest witness $Y \in \mathcal{L}$, valid pre-signatures with respect to Y can be adapted into valid signatures, pre-verify soundness protects against adversaries using malicious witnesses by ensuring that for $Y \notin \mathcal{L}$, it is difficult to find valid pre-signatures with respect to Y . Gerhart *et al.* define two notions of pre-verify soundness:

Definition 2.10 (Computational Pre-verify Soundness [16, Definition 15]). *An adaptor signature scheme AS is computationally pre-verify sound if*

UniqueExtractability _{$\mathcal{A}, \text{AS}$} (λ)	$\mathcal{O}_S(m)$
101 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$	201 : $\sigma \leftarrow \text{Sign}(\text{sk}, m)$
102 : $(m, Y, \hat{\sigma}, \sigma, \sigma') \leftarrow \mathcal{A}^{\mathcal{O}_S, \mathcal{O}_{\text{PS}}(\cdot, \cdot)}(\text{pk})$	202 : return σ
103 : assert $(\sigma' \neq \sigma)$	
104 : assert $\text{Verify}(\text{pk}, m, \sigma) = \text{Verify}(\text{pk}, m, \sigma') = 1$	$\mathcal{O}_{\text{PS}}(m, Y)$
105 : assert $\text{PreVerify}(\text{pk}, m, Y, \hat{\sigma}) = 1$	301 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
106 : $y = \text{Ext}(\sigma, \hat{\sigma}, Y)$	302 : return $\hat{\sigma}$
107 : $y' = \text{Ext}(\sigma', \hat{\sigma}, Y)$	
108 : return $(Y, y) \in R \wedge (Y, y') \in R$	

Fig. 6: The unique extractability game.

for every PPT adversary $\mathcal{A}$ there exists a negligible function ν such that for all $\lambda \in \mathbb{N}$ and all (polynomially-bounded) $Y \notin \mathcal{L}$,

$$\mathbb{P} \left[\text{PreVerify}(\text{pk}, m, \hat{\sigma}, Y) = 1 \mid \begin{array}{l} (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(\lambda) \\ (m, \hat{\sigma}) \leftarrow \mathcal{A}(\text{sk}) \end{array} \right] \leq \nu(\lambda).$$

Definition 2.11 (Statistical Pre-verify Soundness [16, Definition 16]). An adaptor signature scheme AS is statistically pre-verify sound if for all $\lambda \in \mathbb{N}$, all (polynomially-bounded) $Y \notin \mathcal{L}$, and all $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$,

$$\mathbb{P} [\exists (m, \hat{\sigma}) : \text{PreVerify}(\text{pk}, m, \hat{\sigma}, Y) = 1] = 0.$$

2.2 Cryptographic Group Actions

Cryptographic group actions were first studied (under the name “hard homogeneous spaces”) by Couveignes [11], before being independently rediscovered by Rostovtsev and Stolbunov [24] and later formalized and modernized in [2]. These group actions generalize Diffie-Hellman and discrete logarithm-based protocols, and can be used to design similar protocols that are believed to resist attacks by quantum computers. For the purposes of designing digital signatures, we are particularly interested in *effective* group actions, as defined in [2, Definition 3.4]. In this work, we only consider CSI-FiSh, although many of our constructions work for other cryptographic group actions, like those used in LESS and MEDS. Unfortunately, certain protocol optimizations (unrelated to the cryptographic group action framework) used in LESS and MEDS make them incompatible with our adaptor signature mechanism. Thus, since we are interested in protocols that do not require modifications to the underlying digital signature scheme, we restrict our attention to CSI-FiSh. Moreover, since the particulars of the group action used in CSI-FiSh are not important to our construction we omit them; we refer the reader to [19, Sections 2.2.8 and 3.4] for more details.

Hard Relations and Sigma Protocols from Group Actions. Given the action of a group G on a set X , we are interested in relations of the following form,

for any fixed $x_0 \in X$: $\mathcal{R}_{G,x_0} = \{(Y, y) \in X \times G : Y = y * x_0\}$. The corresponding language is $\mathcal{L}_{G,x_0} = \{Y : \exists y \in G \text{ such that } Y = y * x_0\}$. When uniform sampling from G is feasible—in particular, for CSI-FiSh—there is a natural sampling algorithm for the relation $\mathcal{R}_{G,x_0}$: sample $g \leftarrow \$ G$ and set $Y = g * x_0$. Equipped with this sampling algorithm $\mathcal{R}_{G,x_0}$ is a hard relation under the *group action inverse assumption* for G, X :

Definition 2.12 (Group Action Inverse Problem). *Let G act on X , and let $x_0 \in X$. The group action inverse problem (GAIP) is, given $x_1 \in G * x_0$, to find $g \in G$ such that $x_1 = g * x_0$. The group action inverse assumption is that there exists no PPT adversary that solves the GAIP with non-negligible probability.*

One can use multiple public keys to reduce the soundness error of a sigma protocol. This idea naturally leads to the “multi-target” problem.

Definition 2.13 (Multi-Target GAIP). *Let G act on X , and let $x_0 \in X$. The multi-target group action inverse problem (MT-GAIP) is, given $x_1, \dots, x_t \in G * x_0$, to find $g \in G$ such that $x_i = g * x_j$ for some $i, j \in \{0, \dots, t\}$ with $i \neq j$. The multi-target group action inverse assumption is that there exists no PPT adversary that solves the MT-GAIP with non-negligible probability.*

Any cryptographic group action yields a straightforward sigma protocol for the relation $\mathcal{R}_{G,x_0}$ as depicted in Figure 7. The signature scheme in Figure 8 results from applying the Fiat-Shamir transform to this sigma protocol, repeated over several rounds to reach cryptographic levels of soundness and optimized by using multiple public keys and a fixed-weight hash function.

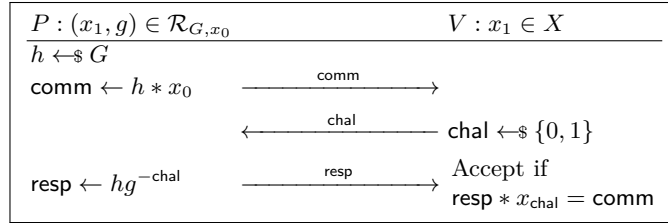


Fig. 7: The canonical sigma protocol associated to $\mathcal{R}_{G,x_0}$.

3 Our Adaptor Signature Construction: ORCAS

Recall the standard construction of a digital signature Σ from CSI-FiSh (or any cryptographic group action) [8], depicted in Figure 8 along with a variant, Filtered CSI-FiSh, which we will use for our security proof.

Notation. Let $S - 1$ be the number of public keys, t the number of rounds, and H a hash function (modeled as a random oracle) with outputs sampled uniformly $\{1 - S, \dots, 0, \dots, S - 1\}^t$. Let $\mathcal{D}_H$ be the distribution of the hash outputs.

We will consider a transitive action of a group G on a set X with starting element $x_0 \in X$, for which GAIP is hard. Denote a public key by $\text{pk} = (x_1, \dots, x_{S-1}) \in X^{S-1}$, with corresponding secret key $\text{sk} = (\text{sk}_1, \dots, \text{sk}_{S-1}) \in G^{S-1}$ such that $x_i = \text{sk}_i * x_0$ for each i . Define $\text{sk}_0 = e_G$ so that $\text{sk}_0 * x_0 = x_0$.

KeyGen(1^λ)	Sign(sk, m)
101 : for $i = 1, \dots, S - 1$:	201 : parse $\text{sk} = (\text{sk}_1, \dots, \text{sk}_{S-1})$
102 : $\text{sk}_i \leftarrow \$ G$	202 : $g_1, \dots, g_t \leftarrow \$ G$
103 : $x_i \leftarrow \text{sk}_i * x_0$	203 : for $i = 1, \dots, t$:
104 : $\text{sk} \leftarrow (\text{sk}_1, \dots, \text{sk}_{S-1})$	204 : $\text{comm}_i = g_i * x_0$
105 : $\text{pk} \leftarrow (x_1, \dots, x_{S-1})$	205 : $\text{chal} = H(m \text{comm}_1 \dots \text{comm}_t)$
106 : return (sk, pk)	206 : parse $\text{chal} = (\text{chal}_1, \dots, \text{chal}_t)$
Verify (pk, m, σ)	207 : $r \leftarrow \$ [t]$
301 : parse $\text{pk} \leftarrow (x_1, \dots, x_{S-1})$	208 : if $\text{chal}_r \neq 0$:
302 : parse $\sigma \leftarrow (\text{chal}, \text{resp}_1, \dots, \text{resp}_t)$	209 : goto line 202
303 : parse $\text{chal} \leftarrow (\text{chal}_1, \dots, \text{chal}_t)$	210 : for $i = 1, \dots, t$:
304 : for $i = 1, \dots, t$:	211 : $\text{resp}_i \leftarrow g_i \cdot \text{sk}_{\text{chal}_i}^{-1}$
305 : $\text{comm}'_i \leftarrow \text{resp}_i * x_{\text{chal}_i}$	212 : return $\sigma = (\text{chal}, \text{resp}_1, \dots, \text{resp}_t)$
306 : $\text{chal}' \leftarrow H(m \text{comm}'_1 \dots \text{comm}'_t)$	
307 : return $\text{chal}' \stackrel{?}{=} \text{chal}$	

Fig. 8: The CSI-FiSh signature scheme. Lines in gray are not included in standard CSI-FiSh but give a variant, **Filtered CSI-FiSh**, in which the probability of a signature being output is proportional to the number of zero entries in its challenge vector. This variant will appear in our proofs in later sections.

Existing Group Action-Based Adaptor Signature Schemes. In order to motivate our adaptor signature construction, we first explain an important technique that originated with Schnorr adaptor signatures [23], and which is used in both IAS [27] and adaptor MCSI-FiSh [18]. This technique—while natural and useful for constructing adaptor signatures—has a fundamental limitation in the group action setting, since it loses the algebraic structure of the discrete logarithm setting. IAS and adaptor MCSI-FiSh overcome this limitation in different ways, but each comes with a disadvantage: IAS’ expensive NIZK proofs, and adaptor MCSI-FiSh’s additional rounds and non-standard underlying signature. Our method does not suffer from either disadvantage.

First consider the Schnorr signature scheme and the corresponding adaptor signature [16, Section 6.1]. The translation is straightforward: instead of constructing the commitment as $\text{comm} = g^r$, we set $\text{comm} = g^r \cdot Y = g^{r+y}$, construct the challenge chal and response $\widehat{\text{resp}} = r - \text{chal} \cdot \text{sk}$ as usual, and finally define the

pre-signature as $(\text{chal}, \widehat{\text{resp}})$. To pre-verify, we check that $g^{\widehat{\text{resp}}} \cdot \text{pk}^{\text{chal}} \cdot Y = \text{comm}$. To adapt, we set $\text{resp} = \widehat{\text{resp}} + y$, and to extract we set $y = \text{resp} - \widehat{\text{resp}}$.

Tairi *et al.* [27] describe the pitfall that occurs when we apply this idea to the protocol of Figure 8 in a naïve way. For simplicity, we first consider a single public key ($S = 2$) and a single round of signing ($t = 1$). The natural approach is to set $\text{comm} = g * Y$ on line 204, and then define chal and $\widehat{\text{resp}}$ as in Figure 8. However we run into a problem during pre-verification when we try to use the Schnorr technique: we have no way to check for a meaningful relationship between comm , pk and Y , since these now live in a *set* with no group structure. We can check for a relationship between any two of these, *e.g.* that $\text{resp} * x_{\text{chal}} = \text{comm}$, but not between all three. Yet the ability to relate comm , pk , and Y together was essential to pre-verification in the Schnorr adaptor signature.

IAS solves this problem for abelian group actions by including a NIZK proof that relates comm , pk , and Y . In particular, pre-signatures in this scheme have two commitments $\widehat{\text{comm}} = g * x_0, \text{comm} = g * Y$ generated from the same ephemeral randomness g . The challenge is generated as usual by hashing comm , and the response $\widehat{\text{resp}}$ is computed as $g \cdot \text{sk}^{-\text{chal}}$. IAS pre-signatures include one additional component: a “proof of parallelness” for $\widehat{\text{comm}}$ and comm , which proves that the *same* underlying g is used to construct them from x_0 and Y , respectively. The pre-verifier checks that $\widehat{\text{comm}} = \widehat{\text{resp}} * x_{\text{chal}}$ and that the parallelness proof is acceptable. These two checks together replace the Schnorr verification, and guarantee that if we adapt the response as $\text{resp} = \widehat{\text{resp}} \cdot y$, then (for an abelian group action)

$$\text{resp} * x_{\text{chal}} = (\widehat{\text{resp}} \cdot y) * x_{\text{chal}} = y * \widehat{\text{comm}} = g * Y = \text{comm}.$$

This ensures the signature $(\text{comm}, \text{resp})$ will verify according to Figure 8. In practice, IAS uses many rounds, of which only the first is constructed as above to be adapted; the rest are constructed as in CSI-FiSh. Unfortunately, even one proof of parallelness is computationally expensive and large.

Adaptor MCSI-FiSh takes a different approach, noting that when $\text{chal} = 0$ only two set elements need to be compared: comm and Y . This is easily accomplished by checking that $\widehat{\text{resp}} * Y = \text{comm}$; if that check passes we are guaranteed, after adapting, to have $\text{resp} * x_0 = \text{comm}$. However, the unpredictability of the hash function means that no round of “honest” CSI-FiSh can be known in advance to have $\text{chal} = 0$. To solve this, on top of the t rounds in standard CSI-FiSh the authors add a $(t + 1)^{\text{st}}$ round with commitment $\text{comm}_{t+1} = g_{t+1} * Y$, challenge chal_{t+1} which is *always* set to 0, and response $\widehat{\text{resp}}_{t+1} = g_{t+1}$; this $(t + 1)^{\text{st}}$ round can be adapted by setting $\text{resp}_{t+1} = \widehat{\text{resp}}_{t+1} \cdot y = g_{t+1} \cdot y$. While this obviates the need for a NIZK proof, it modifies the underlying signature scheme, rendering it unusable except in blockchains that support a non-standard digital signature.

A Remark on Dichotomic Signatures. In [16, Sections 6–8], Gerhart *et al.* present a generic construction for adaptor signatures from dichotomic signatures. We do not attempt to follow this construction because, unfortunately, it

is not suitable for post-quantum adaptor signatures. Dichotomic signatures require a (quasi-injective) homomorphic one-way function, but such a function—specifically, a truly homomorphic one-way function whose domain is an abelian group—was shown by Alamiati to be impossible in a post-quantum setting: a quantum algorithm can use the function’s homomorphism to break its one-wayness [1, Theorem 4.4.1]. While the construction may be salvageable in the post-quantum setting by using a weaker notion, we do not pursue this approach in this work.

Our Construction. Our approach is similar to adaptor MCSI-FiSh but instead of adding an extra “always zero” round we exploit the fact that, while we cannot guarantee any particular round r to have $\text{chal}_r = 0$, we can expect to have $\text{chal}_r = 0$ once in every $2S - 1$ attempts. We guess a round r which we hope will have $\text{chal}_r = 0$, and construct our commitments as

$$\text{comm}_i = \begin{cases} g_i * x_0 & \text{if } i \neq r \\ g_i * Y & \text{if } i = r \end{cases}$$

We then construct the challenges according to Fiat-Shamir: $(\text{chal}_1, \dots, \text{chal}_t) \leftarrow H(m || \text{comm}_1 || \dots || \text{comm}_t)$. If $\text{chal}_r = 0$, we are in luck: we can construct the responses $(\text{resp}_i)_{i=1}^t$, and the resulting pre-signature $(r, (\text{comm}_i)_{i=1}^t, (\text{resp}_i)_{i=1}^t)$ can later be adapted by setting $\text{resp}_r = \text{resp}_r \cdot y$. If $\text{chal} \neq 0$, we can simply generate fresh commitments and try again. This resembles the way that an adversary can attempt to forge a signature in the Fiat-Shamir paradigm: make a random guess at the challenge, construct a compatible commitment and response, and hope the random oracle will output the challenge that was guessed. The crucial difference, however, is that while a forger must guess the challenge in *every* position—which it can do only with probability $(2S - 1)^t$ —we just need to guess the position of a *single* zero. In particular, for randomly chosen r we will have $\text{chal}_r = 0$ with probability $1/(2S - 1)$. After $2S - 1$ tries, we expect to successfully produce an adaptable pre-signature.

Because our construction works by generating commitments repeatedly—like a cheating signer who does not know the secret key—until we get a favourable challenge in a particular round, we call our construction *One Round “Cheating” Adaptor Signatures*, or ORCAS. There two versions of the scheme: Base ORCAS, which admits easy security proofs, and Fast ORCAS, which is efficient enough to be used in practice. We present Base ORCAS in Figure 9.

Theorem 3.1. *Assuming MT-GAIP is hard for the action of G on X , the adaptor signature Base ORCAS (denoted AS) is a correct, pre-signature adaptable, strongly fully extractable, uniquely extractable, unlinkable, and pre-verify sound adaptor signature for Filtered CSI-FiSh in the random oracle model.*

PreSign(sk, m, Y)	PreVerify(pk, m, Y, $\hat{\sigma}$)
101 : parse sk = (sk ₁ , ..., sk _{S-1}) 102 : do 103 : r ←\$ [t] 104 : g ₁ , ..., g _t ←\$ G 105 : for i = 1, ..., t 106 : comm _i ← $\begin{cases} g_i * x_0 & \text{if } i \neq r \\ g_i * Y & \text{if } i = r \end{cases}$ 107 : chal ← H(m comm ₁ ... comm _t) 108 : parse chal = (chal ₁ , ..., chal _t) 109 : until chal _r = 0 110 : for i = 1, ..., t 111 : resp _i ← g _i · sk _{chal_i} ⁻¹ 112 : return $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$	201 : if Y ∉ $\mathcal{L}$ 202 : return 0 203 : parse pk = (x ₁ , ..., x _{S-1}) 204 : parse $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$ 205 : parse chal = (chal ₁ , ..., chal _t) 206 : for i = 1, ..., t 207 : comm' _i ← $\begin{cases} \widehat{\text{resp}}_i * x_{\text{chal}_i} & \text{if } i \neq r \\ \widehat{\text{resp}}_i * Y & \text{if } i = r \end{cases}$ 208 : chal' ← H(m comm' ₁ ... comm' _t) 209 : return (chal' $\stackrel{?}{=} \text{chal} \wedge \text{chal}_r \stackrel{?}{=} 0$)
Ext($\sigma, \hat{\sigma}, Y$)	Adapt($\hat{\sigma}, y$)
301 : parse $\sigma = (\text{chal}, \text{resp}_1, \dots, \text{resp}_t)$ 302 : parse $\hat{\sigma} = (r, \text{chal}', \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$ 303 : if chal ≠ chal' 304 : return ⊥ 305 : return y = $\widehat{\text{resp}}_r^{-1} \cdot \text{resp}_r$	401 : parse $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$ 402 : parse chal = (chal ₁ , ..., chal _t) 403 : for i = 1, ..., t 404 : resp _i ← $\begin{cases} \widehat{\text{resp}}_i & \text{if } i \neq r \\ \widehat{\text{resp}}_i \cdot y & \text{if } i = r \end{cases}$ 405 : return $\sigma = (\text{chal}, \text{resp}_1, \dots, \text{resp}_t)$

Fig. 9: Base ORCAS.

4 Security of Base ORCAS

4.1 Security for Filtered CSI-FiSh

We will first prove the security of Base ORCAS for the modified scheme Filtered CSI-FiSh introduced in Figure 8. We do not propose this scheme to be used in practice, but because the security of Base ORCAS for Filtered CSI-FiSh implies its security for standard CSI-FiSh.

We will prove Theorem 3.1, showing that ORCAS for Filtered CSI-FiSh is correct, and secure with respect to the the strong security definitions of [12, 15, 16]. Throughout this section let KeyGen, Sign, and Verify denote the components of Filtered CSI-FiSh.

We consider Filtered CSI-FiSh instead of standard CSI-FiSh because our adaptor signature scheme has (strongly) canonical signing with reference to this scheme: we can generate a “correctly-distributed” signature by adapting a pre-signature generated with respect to any statement-witness pair (even an adversarially-chosen one). This property greatly simplifies the remaining proofs. Lemmas 4.1, 4.2, and 4.3 are routine; we defer their proofs to Appendix C.

Lemma 4.1 (Canonical Signing). *AS has strongly canonical signing.*

Lemma 4.2 (Pre-Signature Correctness). *AS is pre-signature correct.*

Lemma 4.3 (Pre-Signature Adaptability). *AS is pre-signature adaptable.*

The proof of Lemma 4.4 is substantially more involved.

Lemma 4.4 (Strong Full Extractability). *Assuming MT-GAIP is hard for the action of G on X and H is a collision-resistant hash-function, the adaptor signature scheme AS satisfies strong full extractability in the random oracle model.*

Proof. We will show that the scheme satisfies extractability. Since AS has canonical signing for Filtered CSI-FiSh, $\mathcal{R}$ is a hard relation (under the GAIP assumption), and (by Lemma 4.5 below) AS satisfies unique extractability, by [12, Theorem 3.4] this implies strong full extractability.

We will construct an algorithm $\mathcal{B}$ to attack MT-GAIP by simulating the security game $\text{Extractability}_{\mathcal{A}, \text{AS}}$. The simulator acts identically to the honest challenger, except it does not know sk and must instead program the random oracle in order to answer pre-signature queries. When asked to pre-sign message m with respect to statement Y , it first samples a random challenge chal and an index r in $[t]$. If $\text{chal}_r \neq 0$, then it repeats the random sampling of chal and r until it gets $\text{chal}_r = 0$. The simulator also samples at random a response $\widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t$. It then computes the (unique) commitment $\text{comm}_1, \dots, \text{comm}_t$ consistent with r , chal and $\widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t$, setting $\text{comm}_r = \text{resp}_r * Y$ and $\text{comm}_i = \widehat{\text{resp}}_i * x_{\text{chal}_i}$ for each $i \neq r$. Finally, it programs the random oracle by setting $H[m || \text{comm}_1 || \dots || \text{comm}_t] = \text{chal}$ and outputs the completed pre-signature $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$.

Naturally random oracle programming fails if the random oracle has already been queried on $m || \text{comm}_1 || \dots || \text{comm}_t$. Since the commitment is sampled at random from $C_1 \sqcup \dots \sqcup C_t$, where $C_r = \{(\text{comm}_1, \dots, \text{comm}_t) \in X^t : \text{chal}_r = 0\}$, this will happen with negligible probability.

Notice that the honest pre-signing algorithm chooses r at random before computing chal , while the simulator samples iteratively at random chal and r until it finds an acceptable pair with $\text{chal}_r = 0$. However, this slight difference does not effect the distribution of the output of $\mathcal{O}_{\text{pS}}$, since both chal and r are sampled at random from $\mathcal{D}_H$ and $[t]$, respectively.

Recall the security game Extractability as described in Figure 5. We now make the reduction from $\text{Extractability}_{\mathcal{A}, \text{AS}}$ to the simulator $\mathcal{B}$.

Assuming the pre-signing oracle does not abort, in which case $\text{chal} \leftarrow \$ \mathcal{D}_H$, we claim the pre-signature $\hat{\sigma}$ output by $\mathcal{O}_{\text{pS}}$ is distributed the same in $\mathbf{G}_1$ as in $\mathbf{G}_0$. Fix m and Y and define $p_n(\hat{\sigma}) = \Pr[\hat{\sigma}' = \hat{\sigma} | \hat{\sigma}' \leftarrow \mathbf{G}_n, \mathcal{O}_{\text{pS}}]$ for $n \in \{0, 1\}$. Since each $\widehat{\text{resp}}_i$ is distributed uniformly over G , for any valid pre-signature $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$ we can write $p_0(\hat{\sigma})$ and $p_1(\hat{\sigma})$ as

$$p_0(\hat{\sigma}) = p_1(\hat{\sigma}) = \frac{1}{|G|^t} \Pr[r' = r \wedge \text{chal}' = \text{chal} \mid \text{chal}'_{r'} = 0 \wedge \text{chal}' \leftarrow \$ \mathcal{D}_H \wedge r' \leftarrow \$ [t]].$$

Since the non-abort distributions are identical, the adversary's probability of success in $\mathbf{G}_1$ is $\Pr[\mathbf{G}_1 = 1] \leq \Pr[\mathbf{G}_0 = 1] + \text{negl}(\lambda)$. Notice that we are not

$\mathcal{B}(x_1, \dots, x_{S-1})$	$\mathcal{O}_{\text{pS}}(m, Y)$
101 : $T, \mathcal{Q} \leftarrow \emptyset$	301 : do
102 : parse $\text{pk} = (x_1, \dots, x_{S-1})$	302 : $\text{chal} \leftarrow \$ \mathcal{D}_{\text{H}}$
103 : $(m, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{pS}}(\cdot, \cdot)}(\text{pk})$	303 : parse $\text{chal} = (\text{chal}_1, \dots, \text{chal}_t)$
104 : assert $\text{Verify}(\text{pk}, m, \sigma) = 1$	304 : $r \leftarrow \$ [t]$
105 : for $(Y, \hat{\sigma}) \in T[m]$	305 : until $\text{chal}_r = 0$
106 : assert $(Y, \text{Ext}(\sigma, \hat{\sigma}, Y)) \notin \mathcal{R}$	306 : $\widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t \leftarrow \$ G$
107 : return (m, σ)	307 : for $i = 1, \dots, t :$
$\mathcal{O}_{\text{H}}(x)$	308 : $\text{comm}_i \leftarrow \begin{cases} \widehat{\text{resp}}_i * x_{\text{chal}_i} & \text{if } i \neq r \\ \widehat{\text{resp}}_i * Y & \text{if } i = r \end{cases}$
201 : $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{x\}$	309 : assert $H[m \text{comm}_1 \dots \text{comm}_t] \neq \perp$
202 : if $H[x] = \perp$	310 : $H[m \text{comm}_1 \dots \text{comm}_t] \leftarrow \text{chal}$
203 : $H[x] \leftarrow \$ \mathcal{D}_{\text{H}}$	311 : $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$
204 : return $H[x]$	312 : $T[m] \leftarrow T[m] \cup \{(Y, \hat{\sigma})\}$
	313 : return $\hat{\sigma}$

Fig. 10: Our MT-GAIP algorithm $\mathcal{B}$ built from an adversary $\mathcal{A}$ which attacks the extractability of our scheme.

$\mathbf{G}_0$	$\mathcal{O}_{\text{H}}(x)$
101 : $T, \mathcal{Q} \leftarrow \emptyset$	201 : $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{x\}$
102 : $\text{H} := [\perp]$	202 : if $H[x] = \perp$
103 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$	203 : $H[x] \leftarrow \$ \mathcal{D}_{\text{H}}$
104 : $(m, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{pS}}(\cdot, \cdot)}(\text{pk})$	204 : return $H[x]$
105 : assert $\text{Verify}(\text{pk}, m, \sigma) = 1$	$\mathcal{O}_{\text{pS}}(m, Y)$
106 : for $(Y, \hat{\sigma}) \in T[m]$	301 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
107 : assert $(Y, \text{Ext}(\sigma, \hat{\sigma}, Y)) \notin \mathcal{R}$	302 : $T[m] \leftarrow T[m] \cup \{(Y, \hat{\sigma})\}$
108 : return 1	303 : return $\hat{\sigma}$

Fig. 11: **Game $\mathbf{G}_0$** : This game corresponds to the original Extractability game, in which the adversary $\mathcal{A}$ is given access to a pre-signature oracle $\mathcal{O}_{\text{pS}}$ and an oracle $\mathcal{O}_{\text{H}}$ and, to win, must produce a valid signature (m, σ) such that $(Y, \text{Ext}(\sigma, \hat{\sigma}, Y)) \notin \mathcal{R}$ for any previous queries to $\mathcal{O}_{\text{pS}}$. Thus $\Pr[\mathbf{G}_0 = 1] = \Pr[\text{Extractability}_{\mathcal{A}, \text{AS}}(\lambda) = 1]$.

using the secret key sk anymore. Therefore we have done the reduction from $\text{Extractability}_{\mathcal{A}, \text{AS}}$ to $\mathcal{B}$.

Suppose there exists a PPT adversary $\mathcal{A}$ winning game $\mathbf{G}_0$ with non-negligible probability. So $\mathcal{A}$ also wins game $\mathbf{G}_1$ with non-negligible probability, which we will call $\epsilon(\lambda)$. This is the probability that, instead of aborting, $\mathcal{B}$ returns a message and signature (m, σ) , where $\sigma = (\text{chal}, \text{resp}_1, \dots, \text{resp}_t)$. We consider two cases, based on whether $\mathcal{A}$ has or has not queried the random oracle $\mathcal{O}_{\text{H}}$ to construct σ . Let $\epsilon_0(\lambda)$ be the probability $\mathcal{B}$ outputs (m, σ) such that $\text{chal} \neq \text{H}(x)$

$\mathbf{G}_1$	$\mathcal{O}_{\text{pS}}(m, Y)$
101 : $T, \mathcal{Q} \leftarrow \emptyset$	301 : do
102 : $H := [\perp]$	302 : $\text{chal} \leftarrow \$ \mathcal{D}_H$
103 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$	303 : parse $\text{chal} = (\text{chal}_1, \dots, \text{chal}_t)$
104 : $(m, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{pS}}(\cdot)}(\text{pk})$	304 : $r \leftarrow \$ [t]$
105 : assert $\text{Verify}(\text{pk}, m, \sigma) = 1$	305 : until $\text{chal}_r = 0$
106 : for $(Y, \hat{\sigma}) \in T[m]$	306 : $\widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t \leftarrow \$ G$
107 : assert $(Y, \text{Ext}(\sigma, \hat{\sigma}, Y)) \notin \mathcal{R}$	307 : for $i = 1, \dots, t :$
108 : return 1	308 : $\text{comm}_i \leftarrow \begin{cases} \widehat{\text{resp}}_i * x_{\text{chal}_i} & i \neq r \\ \widehat{\text{resp}}_i * Y & i = r \end{cases}$
$\mathcal{O}_H(x)$	309 : assert $H[m \text{comm}_1 \dots \text{comm}_t] = \perp$
201 : $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{x\}$	310 : $H[m \text{comm}_1 \dots \text{comm}_t] \leftarrow \text{chal}$
202 : if $H[x] = \perp :$	311 : $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$
203 : $H[x] \leftarrow \$ \mathcal{D}_H$	312 : $T[m] \leftarrow T[m] \cup \{(Y, \hat{\sigma})\}$
204 : return $H[x]$	313 : return $\hat{\sigma}$

Fig. 12: **Game $\mathbf{G}_1$.**

for all $x \in \mathcal{Q}$, and $\epsilon_1(\lambda)$ the probability that $\text{chal} = H(x^*)$ for some $x^* \in \mathcal{Q}$. If $\text{chal} = H(x^*)$ we run $\mathcal{B}$ again with the same randomness but a fresh response $H'(x^*) \leftarrow \mathcal{D}_H$ to the random oracle query x^* . By the Forking Lemma [6, Theorem 1], if $\epsilon_1(\lambda)$ is non-negligible, then with non-negligible probability the second run of $\mathcal{B}$ outputs (m', σ') such that $\text{chal}' = H'(x^*)$.

Suppose both runs of $\mathcal{B}$ output message-signature pairs as desired, with $\text{chal} = H(x^*)$ and $\text{chal}' = H'(x^*)$. Define $\text{comm}_i = \text{resp}_i * x_{\text{chal}_i}$ and $\text{comm}'_i = \text{resp}'_i * x_{\text{chal}'_i}$. Since neither run of $\mathcal{B}$ aborted, we know that $\text{Ver}(\text{pk}, m, \sigma) = \text{Ver}(\text{pk}, m', \sigma') = 1$. This gives us two equalities:

$$\begin{aligned} H(m || \text{comm}_1 || \dots || \text{comm}_t) &= \text{chal} = H(x^*), \\ H'(m' || \text{comm}'_1 || \dots || \text{comm}'_t) &= \text{chal}' = H'(x^*). \end{aligned}$$

Since the probability of a collision is negligible, with overwhelming probability $m || \text{comm}_1 || \dots || \text{comm}_t = m' || \text{comm}'_1 || \dots || \text{comm}'_t = x^*$. Since $\mathcal{D}_H$ is large, with overwhelming probability $H(x^*) \neq H'(x^*)$. So $\text{comm}_i = \text{comm}'_i$ for each round i , but there is at least one round j such that $\text{chal}_j \neq \text{chal}'_j$. We can now compute a path between two distinct public keys x_{chal_j} and $x_{\text{chal}'_j}$. Indeed,

$$\text{resp}_j * x_{\text{chal}_j} = \text{comm}_j = \text{comm}'_j = \text{resp}'_j * x_{\text{chal}'_j}.$$

Therefore $g = \text{resp}_j^{-1} \cdot \text{resp}'_j$ is a solution to MT-GAIP, with $g * x_{\text{chal}'_j} = x_{\text{chal}_j}$.

To complete the proof, we show that $\epsilon_1(\lambda)$ is non-negligible. Since $\epsilon(\lambda) = \epsilon_0(\lambda) + \epsilon_1(\lambda)$ is non-negligible by assumption, it suffices to show that $\epsilon_0(\lambda)$ is negligible; that is, $\mathcal{A}$ cannot win with non-negligible probability unless $\text{chal} = H(x^*)$ for some $x^* \in \mathcal{Q}$.

Suppose $\mathcal{B}$ outputs (m, σ) such that $\text{chal} \neq H(x)$ for all $x \in \mathcal{Q}$. Define $x^* = m || \text{comm}_1 || \dots || \text{comm}_t$. Since $\text{Verify}(\text{pk}, m, \sigma) = 1$, we know that $\text{chal} = H(x^*)$ and thus $x^* \notin \mathcal{Q}$. In fact, we claim that *at the time $\mathcal{A}$ returns (m, σ)* the random oracle is undefined at x^* ($H[x^*] = \perp$). So $H(x^*)$ is sampled from $\mathcal{D}_H$ at the time of verification, meaning $\text{Verify}(\text{pk}, m, \sigma) = 1$ only if $\mathcal{A}$ succeeds in blindly guessing $H(x^*)$, which happens with probability $1/|\mathcal{D}_H|$. Therefore $\epsilon_0(\lambda) \leq 1/|\mathcal{D}_H|$ is negligible.

To prove the claim, suppose for sake of contradiction that $H[x^*] \neq \perp$ when $\mathcal{A}$ returns. Since $\mathcal{A}$ has never queried $\mathcal{O}_H(x^*)$, it must be that $H[x^*]$ was defined during some pre-signing query $\mathcal{O}_{\text{ps}}(m', Y)$. Let $\hat{\sigma} = (r, \text{chal}', \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$ be the pre-signature output by this computation, and let $(\text{comm}'_1, \dots, \text{comm}'_t)$ be the commitment used to generate $\hat{\sigma}$. Assuming $H[x^*]$ was defined here, we must have $x^* = m' || \text{comm}'_1 || \dots || \text{comm}'_t$ and therefore $\text{chal}' = \text{chal} = H(x^*)$. Since $\text{PreVerify}(\text{pk}, m, Y, \hat{\sigma}) = 1$, we know that $\text{chal}'_r = \text{chal}_r = 0$ and thus $\text{resp}_r * x_0 = \text{comm}_r = \text{comm}'_r = \widehat{\text{resp}}_r * Y$. Define $y = \text{Ext}(\sigma, \hat{\sigma}, Y) = \widehat{\text{resp}}_r^{-1} \cdot \text{resp}_r$. So $y * x_0 = Y$, meaning $(Y, y) \in \mathcal{R}$. Since $(\hat{\sigma}, Y) \in T[m]$ this would cause $\mathcal{B}$ to abort during its extraction checks, but we assume that $\mathcal{B}$ does not abort. $\square$

Lemma 4.5 (Unique Extractability). *Assuming H is a collision-resistant hash-function, the adaptor signature scheme AS satisfies unique extractability.*

Proof. Suppose $\mathcal{A}$ outputs a tuple $(m, Y, \hat{\sigma}, \sigma, \sigma')$ that wins the UniqueExtractability game. Expand the (pre-)signatures as follows:

$$\begin{aligned}\sigma &= (\text{chal}, \text{resp}_1, \dots, \text{resp}_t), \\ \sigma' &= (\text{chal}', \text{resp}'_1, \dots, \text{resp}'_t), \\ \hat{\sigma} &= (r, \text{chal}'', \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t).\end{aligned}$$

For each $i \in [t]$, define $\text{comm}_i = \text{resp}_i * x_{\text{chal}_i}$ and $\text{comm}'_i = \text{resp}'_i * x_{\text{chal}'_i}$. Since both extractions succeed and Ext checks that the challenges match, we have $\text{chal} = \text{chal}' = \text{chal}''$. Moreover $\text{chal} = \text{chal}'$ and $\sigma \neq \sigma'$ imply that there exists i such that $\text{resp}_i \neq \text{resp}'_i$, thus $\text{comm}_i \neq \text{comm}'_i$. Since $\text{Verify}(\text{pk}, m, \sigma) = \text{Verify}(\text{pk}, m, \sigma') = 1$, we have found a hash collision:

$$H(m || \text{comm}_1 || \dots || \text{comm}_t) = \text{chal} = \text{chal}' = H(m || \text{comm}'_1 || \dots || \text{comm}'_t).$$

If H is collision resistant, then $\mathcal{A}$ can output a winning tuple $(m, Y, \hat{\sigma}, \sigma, \sigma')$ only with negligible probability. Therefore AS satisfies unique extractability. $\square$

Lemma 4.6 (Unlinkability). *AS satisfies unlinkability.*

Proof. We will show that unlinkability is implied by strongly canonical signing, which we have already shown the scheme to have. Recalling the algorithm $\text{CanonicalSign}_{(Y, y)}$ from the definition of strongly canonical signing, we can rewrite the algorithm Challenge from the security game as follows:

$\text{Challenge}(b, m, (Y, y))$	$\text{CanonicalSign}_{(Y, y)}(\text{sk}, m)$
101 : assert $(Y, y) \in \mathcal{R}$	201 : $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$
102 : $\sigma_0 \leftarrow \text{CanonicalSign}_{(Y, y)}(\text{sk}, m)$	202 : $\sigma \leftarrow \text{Adapt}(\hat{\sigma}, y)$
103 : $\sigma_1 \leftarrow \text{Sign}(\text{sk}, m)$	203 : return σ
104 : return σ_b	

Fig. 13: A reformulation of the Challenge algorithm.

For a scheme with strongly canonical signing, by definition σ_0 is distributed identically to σ_1 for every $(Y, y) \in \mathcal{R}$. Therefore the output of **Challenge** is independent of b , meaning $\mathcal{A}$ succeeds with probability exactly $1/2$. $\square$

Lemma 4.7 (Pre-verify Soundness). *AS has pre-verify soundness.*

Proof. Consider a statement $Y \notin \mathcal{L}$, a keypair (sk, pk) , a message m , and a pre-signature $\hat{\sigma}$. Since the first step of pre-verification is to check whether $Y \in \mathcal{L}$, we are guaranteed that $\text{PreVerify}(\text{pk}, m, Y, \hat{\sigma}) = 0$. $\square$

The Lemmas 4.2–4.7 together prove Theorem 3.1.

4.2 Filtered Signatures

Having proven Base ORCAS correct and secure for Filtered CSI-FiSh, we will define precisely the relationship between Filtered CSI-FiSh and CSI-FiSh and use this relationship to prove Base ORCAS correct and secure for CSI-FiSh. We notice that the two signature schemes have identical key generation and verification; the only difference is in their signing algorithms. Even the signing algorithms are not too different: every Filtered CSI-FiSh signature is also CSI-FiSh signature. In fact, signing in Filtered CSI-FiSh is equivalent to generating a sequence of CSI-FiSh signatures and choosing one to output with probability proportional to the number of zeros in its challenge. We call such a relationship *filtering*. We formally define filtering below, first for a pair of general algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$, then for a pair of signature schemes Σ_1 and Σ_2 .

Definition 4.8 (Filtering). *Let $\text{KGen}(1^\lambda)$ be an algorithm that key pairs (sk, pk) . Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be algorithms that take as input secret keys and public keys, respectively, and common information aux . We call $\mathcal{A}_2$ a perfect (resp. statistical) filtering of $\mathcal{A}_1$ (written $\mathcal{A}_2 \leq \mathcal{A}_1$, resp. $\mathcal{A}_2 \lesssim \mathcal{A}_1$) if there exists a PPT algorithm Filter such that for all $(\text{sk}, \text{pk}) \leftarrow \text{KGen}(1^\lambda)$ and all aux , we have*

1. $\{x \leftarrow \mathcal{A}_1(\text{sk}, \text{pk}, \text{aux})\} \subseteq \{x \leftarrow \mathcal{A}_2(\text{sk}, \text{pk}, \text{aux})\}$.
2. $\mathbb{P}[\text{Filter}(\text{pk}, \text{aux}, x) = 1 | x \leftarrow \mathcal{A}_1(\text{sk}, \text{aux})]$ is non-negligible.
3. The statistical distance between the output distributions of $\mathcal{F}(\text{sk}, \text{pk}, \text{aux})$ and $\mathcal{A}_2(\text{sk}, \text{aux})$ is zero (resp. negligible in λ), where $\mathcal{F}$ is as defined in Figure 14.

Perfect filtering immediately implies a statistical filtering: if $\mathcal{A}_2 \leq \mathcal{A}_1$ then $\mathcal{A}_2 \lesssim \mathcal{A}_1$. We now apply the notion of filtering to signature schemes.

$\mathcal{F}(\text{sk}, \text{pk}, \text{aux})$	
101 :	do
102 :	$x \leftarrow \mathcal{A}_1(\text{sk}, \text{aux})$
103 :	$b \leftarrow \text{Filter}(\text{pk}, \text{aux}, x)$
104 :	until $b = 1$
105 :	return x

Fig. 14: $\mathcal{F}$ is the result of filtering the algorithm $\mathcal{A}_1$.

Definition 4.9 (Filtered Signature). Consider two signature schemes $\Sigma_1 = (\text{KeyGen}, \text{Sign}_1, \text{Verify})$ and $\Sigma_2 = (\text{KeyGen}, \text{Sign}_2, \text{Verify})$. We call Σ_2 a perfect (resp. statistical) filtered signature of Σ_1 if Sign_2 is a perfect (resp. statistical) filtering of Sign_1 . In this case we say Σ_2 perfectly (resp. statistically) filters Σ_2 , and write $\Sigma_2 \leq \Sigma_1$ (resp. $\Sigma_2 \lesssim \Sigma_1$).

As its name suggests, Filtered CSI-FiSh (which we will denote as Σ_2) is a perfect filtered signature of CSI-FiSh (which we will denote as Σ_1). The two schemes' key generation and verification are identical by definition. To show that $\Sigma_2 \leq \Sigma_1$ we define Filter as in Figure 15, making $\text{Sign}(\text{sk}, m)$ identical to $\mathcal{F}(\text{sk}, \text{pk}, m)$. Thus each signature output by Sign_2 must also be an output of Σ_1 .

$\text{Filter}_{\text{CSI-FiSh}}(\text{pk}, m, \sigma)$	
101 :	parse $\sigma = (\text{chal}, \text{resp}_1, \dots, \text{resp}_t)$
102 :	$r \leftarrow \$ [t]$
103 :	return $(\text{chal}_r \stackrel{?}{=} 0)$

Fig. 15: Algorithm $\text{Filter}_{\text{CSI-FiSh}}$ to generate Filtered CSI-FiSh from CSI-FiSh.

Moreover, since each element of a CSI-FiSh signature's challenge is uniform in $\{1 - S, \dots, S - 1\}$, the probability that Filter accepts a signature generated by Sign_1 is non-negligible:

$$\mathbb{P}[\text{Filter}(\text{pk}, m, \sigma) = 1 | \sigma \leftarrow \text{Sign}_1(\text{sk}, m)] = \mathbb{P}\left[\text{chal}_r = 0 \mid \begin{array}{l} \text{chal} \leftarrow \$ \mathcal{D}_H \\ r \leftarrow \$ [t] \end{array}\right] = \frac{1}{2S - 1}.$$

Altogether, we have the following lemma.

Lemma 4.10. *Filtered CSI-FiSh perfectly filters CSI-FiSh.*

We are now in a position to reduce the security of Base ORCAS for CSI-FiSh to its security for Filtered CSI-FiSh, using the language of filtered signatures. Our idea is to show general statements of the form, “If Σ_2 filters Σ_1 and AS satisfies security property X for Σ_2 , then AS satisfies security property X for

Σ_1 .” For some properties we require additional hypotheses, but these are simple and modular. We first give a few definitions used in these additional hypotheses.

First, we must have some guarantees on the security of Σ_1 itself. One can easily construct an insecure scheme Σ_1 which filters to a secure scheme Σ_2 : for example, define Σ_1 to append sk to the signature with probability $1/2$, and define Filter to reject all signatures that contain sk . In this example Σ_2 may be EUF-CMA secure but Σ_1 certainly is not. But EUF-CMA alone is not a strong enough guarantee, since we could still design a scheme Σ_1 for which some of the signatures leak information when combined with pre-signatures (e.g. with probability $1/2$ a signature contains $\text{sk} \oplus \text{sk}'$ for some masking secret key sk' , while a pre-signature always contains sk'). The guarantee we want is simulatability in the random oracle model, as defined below.

Definition 4.11 (Simulatable Signature). *A digital signature scheme $\Sigma = (\text{KGen}, \text{Sign}, \text{Verify})$ is perfectly simulatable (in the random oracle model (ROM)) if there exists a PPT algorithm (which programs the random oracle) $\text{Sim}(\text{pk}, m) \rightarrow (\sigma, H)$, where H is a table of input/output pairs programmed by Sim during its execution, such that for all keypairs $(\text{sk}, \text{pk}) \leftarrow \text{KGen}(\text{pp})$ and all $m \in \{0, 1\}^*$, the following distributions are identical:*

$$\mathcal{D}_0 = \left\{ (\sigma, H) : H \leftarrow [\cdot], \sigma \leftarrow \text{Sign}^{\mathcal{O}_H}(\text{sk}, m) \right\}, \text{ and } \mathcal{D}_1 = \{ (\sigma, H) \leftarrow \text{Sim}(\text{pk}, m) \}$$

where, in $\mathcal{D}_0$, the output H is the table of input/output pairs programmed by the queries that Sign makes in the course of its execution, and in $\mathcal{D}_1$, it is the table of input/output pairs programmed by Sim in its execution.

In our security reduction we will have to program the random oracle to simulate signing queries while for other queries using the random oracle provided by the security game we are reducing to. So we want some restrictions on how the algorithms interact with the random oracle, in order to prevent unwanted interference between the programmed random oracle and the provided random oracle. The relevant restrictions are defined below.

Definition 4.12 (Clean Simulation). *A simulator Sim for a signature scheme σ (satisfying Definition 4.11) (in the ROM) is called clean if for all $(\text{sk}, \text{pk}) \leftarrow \text{KGen}(\text{pp})$, and all messages m , if $(\sigma, H) \leftarrow \text{Sim}(\text{pk}, m)$ and $H[x] \neq \perp$ (i.e., Sim programmed the random oracle value $H(x)$ during its execution) then $\text{Verify}^{\mathcal{O}_H}(\text{pk}, m, \sigma)$ queries $\mathcal{O}_H$ at x . We call Σ a cleanly simulatable signature.*

Definition 4.13 (Unpredictable Queries). *A randomized algorithm $\mathcal{A}^{\mathcal{O}_H}$ (in the ROM) has unpredictable queries if for all inputs inputs and all strings $x \in \{0, 1\}^*$, we have $\mathbb{P}[\mathcal{A}^{\mathcal{O}_H}(\text{inputs}) \text{ queries } x] = \text{negl}(\lambda)$, where $\lambda = |\text{inputs}|$.*

Definition 4.14 (Distinct Queries). *A randomized algorithm $\mathcal{A}^{\mathcal{O}_H}$ (in the ROM) has the distinct queries property if for all pairs $\text{inputs}_1 \neq \text{inputs}_2$,*

$$\mathbb{P}[\mathcal{A}^{\mathcal{O}_H}(\text{inputs}_1) \text{ queries } x \wedge \mathcal{A}^{\mathcal{O}_H}(\text{inputs}_2) \text{ queries } x] = 0 \text{ for all } x \in \{0, 1\}^*.$$

Lemma 4.15 will allow us to deduce many security properties of an adaptor signature scheme for a signature scheme from its security for a filtering of that signature scheme. Note that the lemma does not reference unique extractability: this implication does not hold in general. We prove it for ORCAS separately.

Lemma 4.15. *Let Σ_1 and Σ_2 be two signature schemes with $\Sigma_2 \lesssim \Sigma_1$, and let AS be an adaptor signature scheme for Σ_2 with respect to hard relation $\mathcal{R}$. Then:*

1. *If AS is pre-signature correct for Σ_2 , then it is also for Σ_1 .*
2. *If AS is pre-signature adaptable for Σ_2 , then it is also for Σ_1 .*
3. *If AS is statistically (resp. computationally) pre-verify sound for Σ_2 , then AS is statistically (resp. computationally) pre-verify sound for Σ_1 .*

In the random oracle model, if Σ_1 is simulatable and both Sign_1 and PreSign have unpredictable queries, then the following also hold:

4. *If AS is weakly unlinkable for Σ_2 , then it is also for Σ_1 .*
5. *If Σ_1 is cleanly simulatable, $\text{Verify}(\text{pk}, \cdot, \cdot)$ has distinct queries for all pk , and AS satisfies full extractability (resp. strong full extractability) for Σ_2 , then AS satisfies full extractability (resp. strong full extractability) for Σ_1 .*

Proof. Claims 1-3 follow immediately since the algorithm Sign (the only difference between Σ_1 and Σ_2) is irrelevant for pre-signature correctness, pre-signature adaptability, and pre-verify soundness.

To prove claims 4 and 5, we will reduce the relevant security game for Σ_1 to its counterpart for Σ_2 by answering signing queries using the simulator for Σ_1 and pre-signing queries (as well as challenge queries, in the case of weak unlinkability) using the interfaces provided by the security game for Σ_2 . This reduction requires us to stitch together a hash table with some entries given by the outputs of the actual random oracle and others by the signing simulator, but an adversary will not be able to distinguish this from an honest random oracle. We demonstrate the reduction, using a series of hybrid games, for full extractability; the reductions for strong full extractability and weak unlinkability follow the same pattern.

Game G_0 : The original security game $\text{FullExtractability}_1$ for AS and Σ_1 . Let H_1 denote the challenger's random oracle and H_1 its hash table.

Game G_1 : Every time Sign_1 or PreSign would make a random oracle query $H_1(x)$, it first checks whether $H_1[x] = \perp$ and aborts if not (*i.e.*, if the random oracle has already been queried at x).

Game G_2 : Replace Sign_1 with the signing simulator Sim for Σ_1 . Each time Sim outputs (σ, H') , for all $x \in H'$ it checks whether $H_1[x]$ has already been set, aborts if so, but if not sets $H_1[x] = H'[x]$.

Game G_3 : PreSign uses its own random oracle H_2 and hash table H_2 . Each time PreSign makes a random oracle query $H_2(x)$ it checks whether $H_1[x]$ has already been set, aborts if so, but if not sets $H_1[x] = H_2[x]$.

Game G_4 : Each time PreSign makes a query $H_2(x)$, it aborts if $H_1[x]$ has already been set, but does not immediately update it otherwise. Each time $H_1(x)$ is queried, if $H_1[x] = \perp$ it queries $h \leftarrow H_2(x)$ and sets $H_1[x] = h$.

Game $\mathbf{G}_5$: Remove the aborts. Each time Sim outputs (σ, H') , for all $x \in H'$ it sets $H_1[x] = H'[x]$, regardless of whether $H_1[x] = \perp$.

Because Sign_1 and PreSign have unpredictable queries, the probability that a single execution of Sign or PreSign queries $H_1(x)$ for any individual $x \in H_1$ which has already been set is $p \leq \text{negl}(\lambda)$. If an execution of game $\mathbf{G}_1$ has q_S signing queries, q_{PS} pre-signing queries, and q_H (total) random oracle queries, by the union bound the probability of an abort is $p_{\text{abort}} \leq (q_S + q_{PS})q_H \cdot p$. Since a PPT adversary can cause only polynomially many queries to be made, the probability of an abort in game $\mathbf{G}_1$ is negligible. So games $\mathbf{G}_0$ and $\mathbf{G}_1$ are computationally indistinguishable.

Games $\mathbf{G}_1$ and $\mathbf{G}_2$ are identical because the outputs (σ, H') of Sim are distributed identically to those of Sign_1 as long as neither algorithm queries the random oracle on a value x for which $H_1[x]$ is already set. But in this case both games abort, so their behavior is in fact identical.

Games $\mathbf{G}_2$ and $\mathbf{G}_3$ are identical because, as long as $H_1[x]$ has not already been set, querying $h \leftarrow H_2(x)$ then setting $H_2[x] = h$ is equivalent to simply querying $h \leftarrow H_1(x)$. But if $H_1[x]$ has been set, both games abort.

Game $\mathbf{G}_4$ differs from game $\mathbf{G}_3$ only in the delay between setting $H_2[x]$ and updating $H_1[x]$ with it. If $H_1(x)$ is queried while $H_1[x] = H_2[x] = \perp$, then setting $H_1[x]$ by querying $H_2(x)$ is equivalent to sampling $H_1[x]$ from $\mathcal{D}_H$ directly. If $H_2[x]$ is set by PreSign and $H_1(x)$ is queried before Sim attempts to program $H_1[x]$, then the outputs are likewise equivalent. The outputs would only differ if $H_2[x]$ were set by PreSign then Sim attempted to program $H_1[x]$. In game $\mathbf{G}_3$, since $H_1[x]$ was already updated with $H_2[x]$, the simulator would abort. In game $\mathbf{G}_4$, since $H_1[x]$ was not yet updated with $H_2[x]$, the simulator would successfully program $H_1[x] = H'[x]$. But the probability of Sim programming a $H'[x]$ for which $H_2[x] \neq \perp$ is negligible, since Sign_1 has unpredictable queries and Sim has the same distribution of queries as Sign_1 . So games $\mathbf{G}_3$ and $\mathbf{G}_4$ are computationally indistinguishable.

Games $\mathbf{G}_4$ and $\mathbf{G}_5$ differ only when Sim tries to program or PreSign tries to query a value x for which $H_1[x]$ is already defined. But we have already established that, because these algorithms have unpredictable queries, this happens with negligible probability so the games are computationally indistinguishable.

Consider a PPT adversary $\mathcal{A}$ which wins the game $\text{FullExtractability}_1$ for Σ_1 with non-negligible probability $\epsilon(\lambda)$. By the hybrid argument above, $\mathcal{A}$ also wins game $\mathbf{G}_5$ with probability $\epsilon'(\lambda) \geq \epsilon(\lambda) - \text{negl}(\lambda)$. We define a reduction $\mathcal{B}_{\text{FullExtractability}}$ as shown in Figure 16: given an instance pk of $\text{FullExtractability}_2$ for σ_2 , we initiate game $\mathbf{G}_5$ with $\mathcal{A}$ on input pk . While $\text{FullExtractability}_2$ has its tables C_2, T_2, S_2, U_2 , and H_2 , we will also keep of our own tables C_1, T_1, S_1, U_1 , and H_1 . We will answer queries to $\mathcal{O}_{PS}$ and $\mathcal{O}_{\text{NewY}}$ using the oracles provided by $\text{FullExtractability}_2$, updating T_1 and C_1 appropriately. This means we will always have $C_1 = C_2$ and $T_1 = T_2$. We will answer $\mathcal{O}_S(m)$ queries using $(\sigma, H') \leftarrow \text{Sim}(\text{pk}, m)$, updating S_1 and U_1 appropriately and then setting $H_1[x] = H'[x]$ for all $x \in H'$. This means we will always have $S_2 = U_2 = \emptyset$, since $\mathcal{O}_{S_2}$ is never called. For queries to $\mathcal{O}_{H_1}(x)$, if $H_1[x]$ has not already been

set we will set it by querying $\mathcal{O}_{H_2}(x)$, then we will return $H_1[x]$. This behavior is all identical to game $\mathbf{G}_5$. On receiving output from $\mathcal{A}$, we simply pass it on as our output to $\text{FullExtractability}_2$.

$\mathcal{B}_{\text{FullExtractability}}^{\mathcal{O}_{\text{NewY}_2}, \mathcal{O}_{S_2}, \mathcal{O}_{pS_2}, \mathcal{O}_{H_2}}(\text{pk})$		$\mathcal{O}_{S_1}(m)$
101 : $C_1, T_1, S_1, U_1, H_1 \leftarrow \emptyset$		201 : $(\sigma, H') \leftarrow \text{Sim}(\text{pk}, m)$
102 : $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{NewY}_1}, \mathcal{O}_{S_1}, \mathcal{O}_{pS_1}, \mathcal{O}_{H_1}}(\text{pk})$		202 : $S_1 \leftarrow S_1 \cup m$
103 : assert $\text{Verify}^{\mathcal{O}_{H_1}}(\text{pk}, m^*, \sigma^*) = 1$		203 : $U_1 \leftarrow U_1 \cup \{(m, \sigma)\}$
104 : assert $m^* \notin S_1$		204 : for $x \in H'$:
105 : assert $(m^*, \sigma^*) \notin U_1$		205 : $H_1[x] \leftarrow H'[x]$
106 : return (m^*, σ^*)		206 : return σ
$\mathcal{O}_{pS_1}(m, Y)$	$\mathcal{O}_{\text{NewY}_1}(\cdot)$	$\mathcal{O}_{H_1}(x)$
301 : $\hat{\sigma} \leftarrow \mathcal{O}_{pS_2}(m, Y)$	401 : $Y \leftarrow \mathcal{O}_{\text{NewY}_2}(\cdot)$	501 : if $H_1[x] = \perp$:
302 : $T_1[m] \leftarrow T_1[m] \cup \{(Y, \hat{\sigma})\}$	402 : $C_1 \leftarrow C_1 \cup \{Y\}$	502 : $h \leftarrow \mathcal{O}_{H_2}(x)$
303 : return $\hat{\sigma}$	403 : return Y	503 : $H_1[x] \leftarrow h$
		504 : return $H_1[x]$

Fig. 16: Reduction $\mathcal{B}_{\text{FullExtractability}}$ from $\mathbf{G}_5$ for Σ_1 to $\text{FullExtractability}_2$ for Σ_2 .

For weak unlinkability, the reduction is complete since Chall_1 queries are being passed directly to Chall_1 , so $\mathcal{A}$ is guessing the bit b from $\text{WeakUnlinkability}_2$. For (strong) full extractability, we must also ensure that the signature output by the adversary passes all the checks applied by $\text{FullExtractability}_2$. Since $C_1 = C_2$, $T_1 = T_2$, and $U_2 = S_2 = \emptyset$, a valid forgery for $\text{FullExtractability}_1$ will automatically pass every test for $\text{FullExtractability}_2$ except possibly verification with respect to the untampered hash function H_2 . When Σ_1 is cleanly simulatable and $\text{Verify}(\text{pk}, \cdot, \cdot)$ has distinct queries, any forgery which verifies with respect to the modified hash function must also verify with respect to the original.

Assume (m^*, σ^*) is a valid forgery for $\text{FullExtractability}_1$. Let (σ_j, H'_j) be the outputs of the j -th call to the signing simulator, $\text{Sim}(m_j)$. Since $(m^*, \sigma^*) \notin U_2$, we know that $(m^*, \sigma^*) \neq (m_j, \sigma_j)$ for all j . For any $x \in H'_j$, since Sim is clean, $\text{Verify}^{\mathcal{O}_H}(\text{pk}, m_j, \sigma_j)$ queries $\mathcal{O}_H(x)$, and since $\text{Verify}(\text{pk}, \cdot, \cdot)$ has distinct queries, $\text{Verify}^{\mathcal{O}_H}(\text{pk}, m^*, \sigma^*)$ does *not* query $\mathcal{O}_H(x)$. So the only random oracle queries made by $\text{Verify}^{\mathcal{O}_H}(\text{pk}, m^*, \sigma^*)$ are for $x \notin \left(\bigcup_j H'_j\right)$, and for those x we have $H_1[x] = H_2[x]$, so $\text{Verify}^{\mathcal{O}_{H_2}}(\text{pk}, m^*, \sigma^*) = \text{Verify}^{\mathcal{O}_{H_1}}(\text{pk}, m^*, \sigma^*) = 1$. $\square$

4.3 Security for Standard CSI-FiSh

We can now prove Base ORCAS secure for standard CSI-FiSh.

Theorem 4.16. *Assuming MT-GAIP is hard for the action of G on X , Base ORCAS (denoted AS) is a correct, pre-signature adaptable, strongly fully extractable, uniquely extractable, unlinkable, and pre-verify sound adaptor signature for CSI-FiSh in the random oracle model.*

Proof. Denote CSI-FiSh by Σ_1 and Filtered CSI-FiSh by Σ_2 . We know from lemma 4.10 that $\Sigma_1 \leq \Sigma_2$, and from Theorem 3.1 that AS satisfies the above properties for Σ_2 . Furthermore, PreSign and Sign₁ both have unpredictable queries (since the commitment is sampled uniformly at random from an exponentially large space) and Verify(pk, ·, ·) has distinct queries for all pk (since CSI-FiSh verification computes a single hash $H(m||\text{comm}'_1||\dots||\text{comm}'_t)$ whose input depends on both the message and the signature). If Σ_1 is cleanly simulatable, then Lemma 4.15 gives us every security property except unique extractability. So for these properties it suffices to show that Σ_1 is cleanly simulatable, which we do in Lemma 4.17 below.

For unique extractability, we notice that the proof of Lemma 4.5 relied only on the structure of (pre-)signatures and the collision-resistance of H , making no reference to the signature scheme Σ_2 itself. So the same proof shows that AS satisfies unique extractability for Σ_1 . $\square$

Lemma 4.17. *CSI-FiSh is cleanly simulatable.*

Proof. Consider the simulator Sim for CSI-FiSh defined in Figure 17, and compare its output distribution with that of the CSI-FiSh signing algorithm Sign.

Sign(sk, m)	Sim(pk, m)
101 : parse sk = (sk ₁ , ..., sk _{S-1})	201 : parse pk = (x ₁ , ..., x _{S-1})
102 : g ₁ , ..., g _t ←\$ G	202 : resp ₁ , ..., resp _t ←\$ G
103 : for i = 1, ..., t :	203 : chal ←\$ D _H
104 : comm _i ← g _i * x ₀	204 : parse chal = (chal ₁ , ..., chal _t)
105 : chal = H(m comm ₁ ... comm _t)	205 : for i = 1, ..., t :
106 : parse chal = (chal ₁ , ..., chal _t)	206 : comm _i ← resp _i * x _{chal_i}
107 : for i = 1, ..., t :	207 : σ = (chal, resp ₁ , ..., resp _t)
108 : resp _i ← g _i · sk ⁻¹ _{chal_i}	208 : H[m comm ₁ ... comm _t] ← chal
109 : return σ = (chal, resp ₁ , ..., resp _t)	209 : return (σ, H)

Fig. 17: Actual and simulated CSI-FiSh signing algorithms.

For a fixed keypair (sk, pk) and message m , in both cases σ is uniquely determined by (comm₁, ..., comm_t) and chal, distributed independently and uniformly over their respective spaces (X^t for the commitment vector and D_H for the challenge). So Sim is a simulator for CSI-FiSh. It is a clean simulator because it programs exactly one value of its hash table, $H[m||\text{comm}_1||\dots||\text{comm}_t]$, which is precisely the value Verify(pk, m, σ) must query. $\square$

5 Optimization

The most time-intensive part of CSI-FiSh signing is computing the isogenies to find the commitment curves. Since this is the part which Base ORCAS pre-signing must successively recompute until it finds a challenge with $\text{chal}_r = 0$, the expected runtime of **PreSign** is approximately $2S - 1$ times that of **Sign**. Even for relatively few public keys, this is impractical. But we do not actually need to recompute all the commitments from scratch: once we have generated one commitment vector we can rerandomize by permuting the commitments until we receive a favorable challenge. We refer to the optimized scheme as Fast ORCAS, and define it formally in Figure 18.

PreSign(sk, m, Y)	
101 : parse sk = (sk ₁ , ..., sk _{S-1})	110 : while chal _r ≠ 0
102 : r ←\$ [t]	111 : count ← count + 1
103 : g ₁ , ..., g _t ←\$ G	112 : if count > t! : return ⊥
104 : for i = 1, ..., t	113 : ϕ ←\$ S _t
105 : comm _i ← $\begin{cases} g_i * x_0 & \text{if } i \neq r \\ g_i * Y & \text{if } i = r \end{cases}$	114 : r ← ϕ(r)
106 : $\overrightarrow{\text{comm}} \leftarrow (\text{comm}_1, \dots, \text{comm}_t)$	115 : $\overrightarrow{\text{comm}} \leftarrow \phi(\overrightarrow{\text{comm}})$
107 : chal ← H(m comm ₁ ... comm _t)	116 : chal ← H(m comm ₁ ... comm _t)
108 : parse chal = (chal ₁ , ..., chal _t)	117 : parse chal = (chal ₁ , ..., chal _t)
109 : count = 0	118 : for i = 1, ..., t
	119 : $\widehat{\text{resp}}_i \leftarrow g_i \cdot \text{sk}_{\text{chal}_i}^{-1}$
	120 : return $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$

Fig.18: Fast ORCAS pre-signing (other algorithms are unchanged from Base ORCAS). Here S_t denotes the symmetric group of permutations on t elements, and for $\phi \in S_t$, we define $\phi(\overrightarrow{\text{comm}}) := (\text{comm}_{\phi^{-1}(1)}, \text{comm}_{\phi^{-1}(2)}, \dots, \text{comm}_{\phi^{-1}(t)})$.

There is a probability $p_\perp$ that $\text{chal}_r \neq 0$ for every permutation $\phi \in S_n$. In this case we have no choice but to abort and try again with a fresh commitment generated from scratch. This probability is given by $p_\perp = \left(\frac{2S-2}{2S-1}\right)^{t!}$. If t is sufficiently small and S is sufficiently large, then $p_\perp$ can be a problematic: for instance, $p_\perp \approx \frac{1}{2}$ when $S = 2^{15}$ and $t = 7$. For the parameters we consider, however, this is not an issue: even with $S = 2^{12}$ and $t = 9$ we have $p_\perp \approx 2^{-64}$. Thus in practice we assume Fast ORCAS pre-signing never aborts.

We must also ensure that permuting preserves adaptability. If $r' = \phi(r)$ and $(\text{comm}'_1, \dots, \text{comm}'_t) = (\text{comm}_{\phi^{-1}(1)}, \dots, \text{comm}_{\phi^{-1}(t)})$, then we have

$$\text{comm}'_{r'} = \text{comm}_{\phi^{-1}(r')} = \text{comm}_{\phi^{-1}(\phi(r))} = \text{comm}_r.$$

Thus, even after permuting, r' still points to the one “adaptable” round of the pre-signature. With this established, every pre-signature output by the optimized

Filter _{ORCAS,k} (pk, m, Y, $\hat{\sigma}$)	
101 : parse pk = ($x_1, \dots, x_{S-1}$)	110 : do
102 : parse $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t)$	111 : count $\leftarrow$ count + 1
103 : parse chal = ($\text{chal}_1, \dots, \text{chal}_t$)	112 : $\phi \leftarrow \$ S_t$
104 : for $i = 1, \dots, t$	113 : $\overrightarrow{\text{comm}}' \leftarrow \phi(\overrightarrow{\text{comm}}')$
105 : $\text{comm}'_i \leftarrow \begin{cases} \widehat{\text{resp}}_i * x_{\text{chal}_i} & \text{if } i \neq r \\ \widehat{\text{resp}}_i * Y & \text{if } i = r \end{cases}$	114 : chal $\leftarrow H(m \text{comm}'_1 \dots \text{comm}'_t)$
106 : chal' $\leftarrow H(m \text{comm}'_1 \dots \text{comm}'_t)$	115 : parse chal = ($\text{chal}_1, \dots, \text{chal}_t$)
107 : $\overrightarrow{\text{comm}}' \leftarrow (\text{comm}'_1, \dots, \text{comm}'_t)$	116 : while $\text{chal}'_{r'} \neq 0$
108 : count $\leftarrow 0$	117 : rand $\leftarrow \$ [k \cdot (2S - 1)]$
	118 : return (rand $\leq$ count)

Fig. 19: Filtering algorithm Filter_{ORCAS,k} to approximate Fast ORCAS using Base ORCAS. The parameter k controls the closeness of the approximation. As above, we define $\phi(\overrightarrow{\text{comm}}) := (\text{comm}_{\phi^{-1}(1)}, \text{comm}_{\phi^{-1}(2)}, \dots, \text{comm}_{\phi^{-1}(t)})$.

algorithm is a possible outputs of the original algorithm. In fact, we will define a notion of filtering for adaptor signatures and show that that Fast ORCAS filters Base ORCAS, a fact which will allow us to prove the security of Fast ORCAS.

5.1 Filtered Adaptor Signatures

We first define formally a filtered adaptor signature.

Definition 5.1 (Filtered Adaptor Signature). *Consider two adaptor signature schemes $\text{AS}_i = (\text{PreSign}_i, \text{PreVerify}, \text{Adapt}, \text{Ext})$, $i = 1, 2$, for the same signature scheme Σ and hard relation $\mathcal{R}$. We call AS_2 a perfect (resp. statistical) filtered adaptor signature of AS_1 if PreSign_2 is a perfect (resp. statistical) filtering of PreSign_1 . In this case we say AS_2 perfectly (resp. statistically) filters AS_1 , and write $\text{AS}_2 \leq \text{AS}_1$ (resp. $\text{AS}_2 \lesssim \text{AS}_1$).*

We claim that this describes the relationship between the optimized and unoptimized ORCAS variants. Intuitively, given the action of S_t on the set of commitments, Fast ORCAS chooses an orbit uniformly at random then chooses uniformly at random one of the acceptable elements within that orbit, whereas Base ORCAS chooses an acceptable commitment uniformly at random from the whole set. If different S_t -orbits contain different numbers of acceptable commitments, the two distributions will be distinct, but a filtering algorithm can approximate one from the other by estimating the number of acceptable commitments in an orbit and biasing its accept probability to compensate. The algorithm is shown in Figure 19.

Lemma 5.2. *Fast ORCAS statistically filters Base ORCAS.*

The proof of Lemma 5.2 is quite involved; we defer it to Appendix B and focus instead on its consequences. In Theorem 4.16 we proved Base ORCAS secure for

standard CSI-FiSh. We will show that Fast ORCAS inherits this security from Base ORCAS, using general statements of the form, “If AS_2 filters AS_1 and AS_1 satisfies security property X for Σ , then AS_2 satisfies property X for Σ .”

Before formalizing these implications, we need one additional definition. An algorithm has unpredictable outputs if any particular value is unlikely to be output; the (unfiltered) pre-signing algorithm must have this property to ensure that a forgery against the filtered algorithm is also a forgery against the unfiltered algorithm in the strong full extractability game.

Definition 5.3 (Unpredictable Output). *A randomized algorithm $\mathcal{A}$ has unpredictable output if for all inputs inputs and all $x^* \in \{0,1\}^*$, we have $\mathbb{P}[x = x^* | x \leftarrow \mathcal{A}(\text{inputs})] = \text{negl}(\lambda)$, where $\lambda = |\text{inputs}|$.*

We now state Lemma 5.4, formalizing the implications hinted at above. Whereas Lemma 4.15 referenced all properties but unique extractability, this lemma references all but weak unlinkability, and for the same reason: the implication is hard to prove in general, but the property is easy to prove directly for the schemes we are using.

Lemma 5.4. *Let AS_1 and AS_2 be two adaptor signature schemes for signature scheme Σ with respect to hard relation $\mathcal{R}$, and suppose $\text{AS}_2 \lesssim \text{AS}_1$. Then:*

1. *If AS_1 is pre-signature correct for Σ , then so is AS_2 .*
2. *If AS_1 is pre-signature adaptable for Σ , then so is AS_2 .*
3. *If AS_1 is statistically (resp. computationally) pre-verify sound for Σ , then AS_2 is statistically (resp. computationally) pre-verify sound for Σ .*
4. *If AS_1 is uniquely extractable for Σ , then so is AS_2 .*
5. *If AS_1 satisfies full extractability for Σ , then so does AS_2 .*
6. *If PreSign_1 has unpredictable outputs and AS_1 satisfies strong full extractability for Σ , then AS_2 satisfies strong full extractability for Σ .*

Proof. Claim 1 follows from the fact that $\{\hat{\sigma} \leftarrow \text{PreSign}_2(\text{sk}, m, Y)\} \subseteq \{\hat{\sigma} \leftarrow \text{PreSign}_1(\text{sk}, m, Y)\}$, for all inputs sk , m , and Y . Any property which holds for all $\hat{\sigma} \leftarrow \text{PreSign}_1(\text{sk}, m, Y)$ must therefore hold for all $\hat{\sigma} \leftarrow \text{PreSign}_2(\text{sk}, m, Y)$. Claims 2 and 3 follow immediately, since the respective definitions make no reference to pre-signing.

For claims 4-6, suppose we have a PPT adversary $\mathcal{A}$ that wins the respective security game $\mathbf{G}_2$ for AS_2 with non-negligible probability $\epsilon(\lambda)$. We use $\mathcal{A}$ to win the same security game $\mathbf{G}_1$ for AS_1 . First, define a hybrid game $\mathbf{G}'_2$ which is identical to $\mathbf{G}_2$ except that it answers pre-signing queries $\mathcal{O}_{\text{ps}_2}(m, Y)$ not by calling $\text{PreSign}_2(\text{sk}, m, Y)$ but by calling the filtering algorithm $\mathcal{F}(\text{sk}, \text{pk}, m, Y)$ defined in Figure 14. Since $\text{PreSign}_2 \lesssim \text{PreSign}_1$, the statistical distance between the output distributions of $\mathcal{O}_{\text{ps}_2}$ in $\mathbf{G}_2$ and in $\mathbf{G}'_2$ is negligible; that is, the games themselves are statistically indistinguishable.

Now we construct a reduction from game $\mathbf{G}_1$ to game $\mathbf{G}'_2$. We answer all of $\mathcal{A}$'s queries except pre-signing queries by passing them to the oracles provided by $\mathbf{G}_1$. We answer pre-signing queries as shown in Figure 20, which is identical to calling $\mathcal{F}(\text{sk}, \text{pk}, m, Y)$. We pass $\mathcal{A}$'s output as our output to $\mathbf{G}_1$.

$\mathcal{O}_{\text{pS}_2}(m, Y)$	
101 :	do
102 :	$\hat{\sigma} \leftarrow \mathcal{O}_{\text{pS}_1}(m, Y)$
103 :	$b \leftarrow \text{Filter}(\text{pk}, m, Y, \hat{\sigma})$
104 :	until $b = 1$
105 :	return $\hat{\sigma}$

Fig. 20: Answering AS_2 pre-signing queries with the AS_1 pre-signing oracle.

For unique extractability and full extractability the proof is now complete, since a valid output for game $\mathbf{G}_2$ is also valid for game $\mathbf{G}_1$. For strong full extractability we must also ensure that the forgery (m^*, σ^*) is not related to any $\hat{\sigma}_i$ produced by game $\mathbf{G}_1$ in response to some query $\mathcal{O}_{\text{pS}_1}(m_i, Y_i)$. Since σ^* is a forgery for game $\mathbf{G}_2$, it cannot be related to any $\hat{\sigma}_i$ that was returned by $\mathcal{O}_{\text{pS}_2}$. The only other possibility is that $\hat{\sigma}_i$ was returned by $\mathcal{O}_{\text{pS}_1}$ in response to a query from $\mathcal{O}_{\text{pS}_2}(m_i, Y_i)$, but $\text{Filter}(\text{pk}, m_i, Y_i, \hat{\sigma}_i)$ returned 0 and thus $\hat{\sigma}_i$ was not returned to $\mathcal{A}$. But $\mathcal{A}$ makes only polynomially-many queries to $\mathcal{O}_{\text{pS}_2}$, and each execution of $\mathcal{O}_{\text{pS}_2}$ makes only polynomially-many queries to $\mathcal{O}_{\text{pS}_1}$, so the total number of queries to $\mathcal{O}_{\text{pS}_1}$ is bounded by some Q polynomial in λ . Since each query $\mathcal{O}_{\text{pS}_1}$ makes to $\mathcal{O}_{\text{pS}_1}$ is independent of each other query, and since PreSign_1 has unpredictable outputs, by the union bound we have

$$\mathbb{P}[\exists i : \hat{\sigma}_i = \hat{\sigma}^*] \leq Q \cdot \mathbb{P}[\hat{\sigma}_i = \hat{\sigma}^* | \sigma_i \leftarrow \text{PreSign}_1(\text{sk}, m_i, Y_i)] \leq Q \cdot \text{negl}(\lambda) \leq \text{negl}(\lambda).$$

Therefore, even for strong full extractability, we win game $\mathbf{G}_1$ with probability at least $\epsilon(\lambda) - \text{negl}(\lambda)$. $\square$

5.2 Security of Fast ORCAS

In Theorem 3.1 we showed the security of an inefficient adaptor signature scheme (Base ORCAS) for a nonstandard signature scheme (Filtered CSI-FiSh). We are now ready to prove our main result: the security of an efficient adaptor signature scheme (Fast ORCAS) for a standard signature scheme (CSI-FiSh).

Theorem 5.5. *Assuming MT-GAIP is hard for the action of G on X , Fast ORCAS (denoted AS_2) is a correct, pre-signature adaptable, strongly fully extractable, uniquely extractable, unlinkable, and pre-verify sound adaptor signature for CSI-FiSh (denoted Σ) in the random oracle model.*

Proof. Observe that the distribution of $(r, \text{comm}_1, \dots, \text{comm}_t)$ is independent of Y for any call to $\text{PreSign}(\text{sk}, m, Y)$. Since both $\hat{\sigma}$ and $\sigma = \text{Adapt}(\hat{\sigma}, y)$ are fully determined by $(\text{comm}_1, \dots, \text{comm}_t)$, and since adapting preserves the commitments, we conclude that the output of $\text{CanonicalSign}_{\text{AS}_2}^{(Y, y)}(\text{sk}, m)$ is independent of Y , and thus AS_2 satisfies weak unlinkability.

For the other properties: Theorem 4.16 shows that Base ORCAS (denoted AS_1) satisfies these properties for Σ . By Lemma 5.2, $AS_2 \lesssim AS_1$, so if $PreSign_1$ has unpredictable outputs we can apply Lemma 5.4 to conclude the proof.

To show that $PreSign_1$ has unpredictable outputs, observe that pre-signatures are in bijection with vectors $(r, comm_1, \dots, comm_t)$ which are sampled uniformly (before rejection sampling) from $[t] \times X^t$. The number of such vectors generated before an acceptable one is found (for which $chal_r = 0$) is bounded by some polynomial M . So the probability of a particular vector $(r, comm_1, \dots, comm_t)$ being generated (much less accepted) is at most $\frac{M}{t|X|^t} \leq \text{negl}(\lambda)$. Thus $PreSign_1$ has unpredictable outputs, completing the proof. $\square$

6 Implementation, Parameters, and Performance

To implement ORCAS, we fork from commit a7ccb87 of the CSI-FiSh reference implementation (<https://github.com/KULEuven-COSIC/CSI-FiSh>), using the eXtended Keccak Code Package for its SHAKE256 implementation as well as the GMP library for high-precision arithmetic. Our implementation is available at <https://github.com/NCDaly/ORCAS-CSI-FiSh>.

We fix $\lambda = 128$ bits of security. Note that the codomain of the hash function of CSI-FiSh is $\{-S+1, \dots, -1, 0, 1, \dots, S-1\}$, thanks to the existence of the twists. While this is the challenge set described in the CSI-FiSh paper, the reference implementation excludes the challenge zero, presumably because it allows for slightly simpler code. We update the challenge set to reflect the paper, shortening signatures by several rounds when there are not too many public keys. Here d is the depth of the pk tree, with public keys $pk_1, \dots, pk_{2^d}$ and thus $S = 2^d + 1$; t is the number of rounds; and k is the rate of the slow hash function.

We performed the testing on an Intel Xeon Gold 6246R CPU @ 3.40GHz.

d	t	k	$ sk $	$ pk $	$ \hat{\sigma} $	$ \sigma $	KeyGen	Sig	Ver	PreSig	PreVer	Adapt	Ext
1	49	15	16	128	1650	1649	0.11	2.00	2.01	2.03	2.00	0.005	0.005
2	36	14	16	256	1221	1220	0.19	1.46	1.47	1.49	1.47	0.005	0.005
4	23	12	16	1024	792	791	0.70	0.94	0.95	0.99	0.95	0.005	0.005
6	16	16	16	4096	561	560	2.65	0.68	0.68	3.40	0.68	0.005	0.005
8	13	11	16	16384	462	461	10.39	0.53	0.53	0.85	0.54	0.005	0.005
10	11	7	16	65536	396	395	40.67	0.45	0.45	0.55	0.44	0.005	0.005
12	9	11	16	262144	330	329	164.38	0.37	0.36	6.23	0.36	0.005	0.005

Table 1: ORCAS performance. Sizes are in bytes and times in seconds.

Notice that, with respect to the pre-signature sizes in the first adaptor signature built from CSI-FiSh [27], our scheme's are smaller because we don't have to provide the expensive NIZK proof.

References

1. Alamati, N.: Algebraic Frameworks for Cryptographic Primitives. PhD thesis, University of Michigan, USA (2020). <https://deepblue.lib.umich.edu/handle/2027.42/166137>.
2. Alamati, N., De Feo, L., Montgomery, H., Patranabis, S.: Cryptographic Group Actions and Applications. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020, Part II. LNCS, pp. 411–439. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64834-3_14
3. Aumayr, L., Ersoy, O., Erwig, A., Faust, S., Hostáková, K., Maffei, M., Moreno-Sanchez, P., Riahi, S.: Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part II. LNCS, pp. 635–664. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-92075-3_22
4. Baecker, R., Gerhart, P., Katz, J., Schröder, D.: Fair Exchange for Decentralized Autonomous Organizations via Threshold Adaptor Signatures, Cryptology ePrint Archive, Report 2025/388 (2025). <https://eprint.iacr.org/2025/388>.
5. Battagliola, M., Borin, G., Meneghetti, A., Persichetti, E.: Cutting the GRASS: Threshold GGroup Action Signature Schemes. In: Oswald, E. (ed.) CT-RSA 2024. LNCS, pp. 460–489. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-58868-6_18
6. Bellare, M., Neven, G.: Multi-signatures in the plain public-Key model and a general forking lemma. In: Juels, A., Wright, R.N., De Capitani di Vimercati, S. (eds.) ACM CCS 2006, pp. 390–399. ACM Press (2006). <https://doi.org/10.1145/1180405.1180453>
7. Beullens, W., Katsumata, S., Pintore, F.: Calamari and Falaff: Logarithmic (Linkable) Ring Signatures from Isogenies and Lattices. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020, Part II. LNCS, pp. 464–492. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64834-3_16
8. Beullens, W., Kleinjung, T., Vercauteren, F.: CSI-FiSh: Efficient Isogeny Based Signatures Through Class Group Computations. In: Galbraith, S.D., Moriai, S. (eds.) ASIACRYPT 2019, Part I. LNCS, pp. 227–247. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-34578-5_9
9. BIASSE, J.-F., Micheli, G., Persichetti, E., Santini, P.: LESS is More: Code-Based Signatures Without Syndromes. In: Nitaj, A., Youssef, A.M. (eds.) AFRICACRYPT 20. LNCS, pp. 45–65. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-51938-4_3
10. Chou, T., Niederhagen, R., Persichetti, E., Randrianarisoa, T.H., Reijnders, K., Samardjiska, S., Trimoska, M.: Take Your MEDS: Digital Signatures from Matrix Code Equivalence. In: El Mrabet, N., De Feo, L., Duquesne, S. (eds.) AFRICACRYPT 23. LNCS, pp. 28–52. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-37679-5_2
11. Couveignes, J.-M.: Hard Homogeneous Spaces, Cryptology ePrint Archive, Report 2006/291 (2006). <https://eprint.iacr.org/2006/291>.
12. Dai, W., Okamoto, T., Yamamoto, G.: Stronger Security and Generic Constructions for Adaptor Signatures. In: Isobe, T., Sarkar, S. (eds.) INDOCRYPT 2022. LNCS, pp. 52–77. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-22912-1_3

13. Das, P., Faust, S., Loss, J.: A Formal Treatment of Deterministic Wallets. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019, pp. 651–668. ACM Press (2019). <https://doi.org/10.1145/3319535.3354236>
14. Erwig, A., Riahi, S.: Deterministic Wallets for Adaptor Signatures. In: Atluri, V., Di Pietro, R., Jensen, C.D., Meng, W. (eds.) ESORICS 2022, Part II. LNCS, pp. 487–506. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-17146-8_24
15. Gerhart, P., Schröder, D., Soni, P., Thyagarajan, S.A.K.: Foundations of Adaptor Signatures. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part II. LNCS, pp. 161–189. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-58723-8_6
16. Gerhart, P., Schröder, D., Soni, P., Thyagarajan, S.A.: Foundations of Adaptor Signatures, Cryptology ePrint Archive, Report 2024/1809 (2024). <https://eprint.iacr.org/2024/1809>.
17. Glaeser, N., Maffei, M., Malavolta, G., Moreno-Sanchez, P., Tairi, E., Thyagarajan, S.A.K.: Foundations of Coin Mixing Services. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022, pp. 1259–1273. ACM Press (2022). <https://doi.org/10.1145/3548606.3560637>
18. Jana, P., Shaw, S., Dutta, R.: Compact Adaptor Signature from Isogenies with Enhanced Security. In: Kohlweiss, M., Di Pietro, R., Beresford, A.R. (eds.) CANS 2024, Part I. LNCS, pp. 77–100. Springer, Singapore (2024). https://doi.org/10.1007/978-981-97-8013-6_4
19. Katsumata, S., Lai, Y.-F., LeGrow, J.T., Qin, L.: CSI-Otter: isogeny-based (partially) blind signatures from the class group action with a twist. DCC **92**(11), 3587–3643 (2024). <https://doi.org/10.1007/s10623-024-01441-7>
20. Kuchta, V., LeGrow, J.T., Persichetti, E.: Post-Quantum Blind Signatures from Matrix Code Equivalence, Cryptology ePrint Archive, Report 2025/274 (2025). <https://eprint.iacr.org/2025/274>.
21. Madathil, V., Thyagarajan, S.A.K., Vasilopoulos, D., Fournier, L., Malavolta, G., Moreno-Sanchez, P.: Cryptographic Oracle-based Conditional Payments. In: NDSS 2023. The Internet Society (2023)
22. Miller, A., Bentov, I., Bakshi, S., Kumaresan, R., McCorry, P.: Sprites and State Channels: Payment Networks that Go Faster Than Lightning. In: Goldberg, I., Moore, T. (eds.) FC 2019. LNCS, pp. 508–526. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-32101-7_30
23. Poelstra, A.: Scriptless scripts. Presentation Slides (2017)
24. Rostovtsev, A., Stolbunov, A.: Public-Key Cryptosystem Based On Isogenies, Cryptology ePrint Archive, Report 2006/145 (2006). <https://eprint.iacr.org/2006/145>.
25. Schnorr, C.-P.: Efficient Signature Generation by Smart Cards. Journal of Cryptology **4**(3), 161–174 (1991). <https://doi.org/10.1007/BF00196725>
26. Tairi, E., Moreno-Sanchez, P., Maffei, M.: A²L: Anonymous Atomic Locks for Scalability in Payment Channel Hubs. In: 2021 IEEE Symposium on Security and Privacy, pp. 1834–1851. IEEE Computer Society Press (2021). <https://doi.org/10.1109/SP40001.2021.00111>
27. Tairi, E., Moreno-Sanchez, P., Maffei, M.: Post-Quantum Adaptor Signature for Privacy-Preserving Off-Chain Payments. In: Borisov, N., Díaz, C. (eds.) FC 2021, Part II. LNCS, pp. 131–150. Springer, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-662-64331-0_7

Supplementary Material

A Background and Definitions

A.1 Hard Relations and Non-Interactive Zero-Knowledge

Definition A.1 (Hard Relation [27, Definition 1]). A hard relation $\mathcal{R}$ with underlying language $\mathcal{L}$ is an NP relation such that there is a PPT sampling algorithm GenR that, on input 1^λ , outputs a statement/witness pair (Y, y) , such that for all PPT adversaries A ,

$$\mathbb{P} \left[(Y, y') \in \mathcal{R} \mid \begin{array}{l} (Y, y) \leftarrow \text{GenR}(1^\lambda) \\ y' \leftarrow A(Y) \end{array} \right] = \text{negl}(\lambda),$$

where $\text{negl}(\lambda)$ is a negligible function in the security parameter.

Given $(Y, y) \in \mathcal{R}$ for an NP relation $\mathcal{R}$, it is easy to prove that $Y \in \mathcal{L}$ simply by showing y . In many cases, however, we wish to prove that $Y \in \mathcal{L}$ without leaking any information about y . For this we can use a non-interactive zero-knowledge proof of knowledge.

Definition A.2 (Non-Interactive Zero-Knowledge Proof of Knowledge [27]).

A pair (P, V) of PPT algorithms is called a non-interactive zero-knowledge proof of knowledge (NIZKPoK) with an online extractor for a relation $\mathcal{R}$, random oracle $\mathcal{H}$ and security parameter λ (in the random oracle model) if the following holds:

Completeness: For any $(Y, y) \in \mathcal{R}$ and any $\pi \leftarrow P^{\mathcal{H}}(Y, y)$ there exists a negligible function negl such that it holds that $\mathbb{P}[V^{\mathcal{H}}(Y, \pi) = 1] \geq 1 - \text{negl}(\lambda)$.

Zero Knowledge: There exists a PPT algorithm S , the zero knowledge simulator, such that for any pair (Y, y) and any PPT algorithm D the following distributions are computationally indistinguishable:

1. Let $\pi \leftarrow P^{\mathcal{H}}(Y, y)$ if $(Y, y) \in \mathcal{R}$, and $\pi \leftarrow \perp$ otherwise. Output $D^{\mathcal{H}}(Y, y, \pi)$.
2. Let $\pi \leftarrow S(Y, 1)$ if $(Y, y) \in \mathcal{R}$, and $\pi \leftarrow S(Y, 0)$ otherwise. Output $D^{\mathcal{H}}(Y, y, \pi)$.

Online Extractor: There exist a PPT algorithm K , the online extractor, such that the following holds for any algorithm A . Let $(Y, \pi) \leftarrow A^{\mathcal{H}}(\lambda)$ and H_A be the sequence of queries of A to $\mathcal{H}$ and $\mathcal{H}$'s answers. Let $y \leftarrow K(Y, \pi, H_A)$. Then it holds that

$$\mathbb{P}[(Y, y) \notin \mathcal{R} \wedge V^{\mathcal{H}}(Y, \pi) = 1] \leq \text{negl}(\lambda),$$

where $\text{negl}(\lambda)$ is a negligible function in the security parameter.

A.2 Digital Signatures

Digital signature schemes are cryptographic protocols that enable a user to sign messages in a way that allows anyone to verify the message's authenticity and integrity using the signer's public key, ensuring that the message comes from the claimed sender and has not been tampered with.

Definition A.3 (Digital Signature Scheme). A digital signature scheme is a tuple $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ of algorithms defined as follows:

KeyGen(1^λ): On input a security parameter 1^λ , outputs a keypair (sk, pk) .

Sign(sk, m): On input a secret key sk and a message m , returns a signature σ .

Verify(pk, m, σ): On input a public key pk , a message m , and a signature σ , returns a bit $b \in \{0, 1\}$.

A digital signature scheme Σ is *correct* if honestly-generated signatures always pass verification; more precisely, if for all messages m and all $\lambda \in \mathbb{N}$, we have

$$\mathbb{P} \left[\text{Verify}(\text{pk}, m, \sigma) = 1 \mid \begin{array}{l} (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda) \\ \sigma \leftarrow \text{Sign}(\text{sk}, m) \end{array} \right] = 1.$$

The standard security notion for digital signatures is *existential unforgeability under adaptive chosen message attack*, abbreviated EUF-CMA. Intuitively this says that an efficient adversary who has access to a signing oracle should be unable to produce a message m^* and valid signature σ^* which it has not been signed before. More precisely, we have the following definition:

Definition A.4 (EUF-CMA Security). A signature scheme Σ is EUF-CMA secure if for every PPT adversary $\mathcal{A}$ there exists a negligible function ν such that for all $\lambda \in \mathbb{N}$ $\mathbb{P}[\text{SigForge}_{\mathcal{A}, \Sigma}(\lambda) = 1] \leq \nu(\lambda)$, where the $\text{SigForge}_{\mathcal{A}, \Sigma}$ game is defined in Figure 21.

$\text{SigForge}_{\mathcal{A}, \Sigma}(\lambda)$	$\mathcal{O}_S(m)$
101 : $\mathcal{Q} \leftarrow \emptyset$	201 : $\sigma \leftarrow \text{Sign}(\text{sk}, m)$
102 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$	202 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$
103 : $(m, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}_S}(\text{pk})$	203 : return σ
104 : return $(m \notin \mathcal{Q} \wedge \text{Verify}(\text{pk}, m, \sigma))$	

Fig. 21: The $\text{SigForge}_{\mathcal{A}, \Sigma}$ game.

While the previous definition is the standard security notion for digital signatures, it does not address the difficulty of transforming a valid signature of a message m into another valid signature of the same message m . The difficulty of this transformation is captured by a stronger notion of security, called *strong existential unforgeability under chosen message attack*, abbreviated SUF-CMA. In the following we describe how it works.

Definition A.5 (SUF-CMA Security). A signature scheme Σ is SUF-CMA secure if for every PPT adversary $\mathcal{A}$ there exists a negligible function ν such that for all $\lambda \in \mathbb{N}$

$$\mathbb{P}[\text{StrongSigForge}_{\mathcal{A}, \Sigma}(\lambda) = 1] \leq \nu(\lambda),$$

where the $\text{StrongSigForge}_{\mathcal{A}, \Sigma}$ game is defined in Figure 22.

$\text{StrongSigForge}_{\mathcal{A}, \Sigma}(\lambda)$	$\mathcal{O}_S(m)$
101 : $\mathcal{Q} \leftarrow \emptyset$	201 : $\sigma \leftarrow \text{Sign}(\text{sk}, m)$
102 : $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$	202 : $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(m, \sigma)\}$
103 : $(m, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}_S}(\text{pk})$	203 : return σ
104 : return $((m, \sigma) \notin \mathcal{Q} \wedge \text{Verify}(\text{pk}, m, \sigma))$	

Fig. 22: The $\text{StrongSigForge}_{\mathcal{A}, \Sigma}$ game.

B Fast ORCAS Filters Base ORCAS

Lemma 5.2 states that Fast ORCAS is a statistical filtering of Base ORCAS. We will prove this abstractly, using simplified notation to represent our actual construction. For a security parameter λ , consider a table $V = [m] \times [n]$ with size exponential in λ . For some integer $\ell \leq \text{poly}(\lambda)$, consider a subset $U \subseteq T$ of “acceptable” elements, chosen at random such that for each $(i, j) \in T$ there is an independent probability $1/\ell$ that $(i, j) \in U$. For each $i \in [m]$ define the weight of row i as $\text{wt}(i) = |\{j \in [n] : (i, j) \in U\}|$ and its density $\rho(i) = \frac{\text{wt}(i)}{n}$. Define the total weight $w = |U| = \sum_{i=1}^m \text{wt}(i)$. Define algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$ as in Figure 23, along with the filtering algorithm Filter_k , whose “strength” $k \leq \text{poly}(\lambda)$ controls the closeness of the approximation. Lemma B.2 shows that $k \geq 2\lambda\ell(\lambda + \ln(m))$ is sufficient to ensure the approximation error is negligible.

We claim this setup models our ORCAS construction. The table $T \simeq [t] \times X^t$ is the set of all possible vectors $(r, \overrightarrow{\text{comm}})$, which is in bijection with the set of possible pre-signatures. For a particular message m , the subset $U(m)$ of acceptable elements is the set of all $(r, \overrightarrow{\text{comm}})$ such that $H(m || \text{comm}_1 || \dots || \text{comm}_t)_r = 0$. When H is modeled as a random function with outputs uniform over $\{1 - S, \dots, 0, \dots, S - 1\}^t$, we see that U is distributed correctly with $\ell = 2S - 1$.

Each row i corresponds to an orbit of the action of the symmetric group S_t : i.e., the set $\{(\phi(r), \phi(\overrightarrow{\text{comm}})) : \phi \in S_t\}$ for some representative $(r, \overrightarrow{\text{comm}})$, and each element (i, j) represents a particular vector $(r, \overrightarrow{\text{comm}})$ in that orbit. If we fix a canonical representative $(r_i, \overrightarrow{\text{comm}}_i)$ for each row/orbit i , we may identify each column j with a permutation $\phi_j \in S_t$. Thus (i, j) corresponds to the vector $(\phi_j(r_i), \phi_j(\overrightarrow{\text{comm}}_i))$.

$\mathcal{A}_1(U)$	$\mathcal{A}_2(U)$	$\text{Filter}_k(U, i, j)$
101 : do	201 : do	301 : $c = 0$
102 : $i \leftarrow \$ [m]$	202 : $i \leftarrow \$ [m]$	302 : do
103 : $j \leftarrow \$ [n]$	203 : until $w_i > 0$	303 : $j' \leftarrow \$ [n]$
104 : until $(i, j) \in U$	204 : do	304 : $c \leftarrow c + 1$
105 : return (i, j)	205 : $j \leftarrow \$ [n]$	305 : until $(i, j') \in U$
	206 : until $(i, j) \in U$	306 : $r \leftarrow \$ [k \cdot \ell]$
	207 : return (i, j)	307 : return $(r \leq c)$

Fig. 23: Two algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$ with a filtering algorithm Filter_k . Compare the algorithms PreSign_1 (Figure 9) for Base ORCAS, PreSign_2 (Figure 18) for Fast ORCAS, and their filtering algorithm $\text{Filter}_{\text{ORCAS},k}$ (Figure 19).

The vectors $(r, \overrightarrow{\text{comm}})$ which have $\text{comm}_i = \text{comm}_j$ for some $i \neq j$ are rare enough that we can safely ignore them: if $N = |X|$, when $\overrightarrow{\text{comm}}$ is sampled uniformly from X^t we have

$$\mathbb{P}[\exists i \neq j : \text{comm}_i = \text{comm}_j] = 1 - \prod_{i=1}^{t-1} \frac{N-i}{N} \leq \text{negl}(\lambda).$$

For our purposes, then, each orbit has exactly $n = t!$ elements, and there are exactly $m = t \binom{N}{t}$ different orbits. By comparing the algorithms in Figures 9, 18, and 19 with those in Figure 23, we see that $\text{PreSign}_1 \simeq \mathcal{A}_1$, $\text{PreSign}_2 \simeq \mathcal{A}_2$, and $\text{Filter}_{\text{ORCAS},k} \simeq \text{Filter}_k$. We can therefore prove that $\text{PreSign}_2 \lesssim \text{PreSign}_1$ simply by proving that $\mathcal{A}_2 \lesssim \mathcal{A}_1$.

Lemma B.1. *If $\mathcal{A}_1$, $\mathcal{A}_2$, and Filter_k are defined as in Figure 23, then $\mathcal{A}_2 \lesssim \mathcal{A}_1$.*

Proof. First, observe that Filter_k is efficient with expected runtime of $O(\ell)$. Now we will study the accept probability of Filter_k and output distribution of the filtering algorithm $\mathcal{F}$. Whether $(i, j) \leftarrow \mathcal{A}_1(U)$ or $(i, j) \leftarrow \mathcal{A}_2(U)$, in either case

$$\mathbb{P}[(i, j) = (i_0, j_0) | i = i_0] = \frac{1}{\text{wt}(i)}.$$

So we need only concern ourselves with the distribution of i . On the one hand, if $(i, j) \leftarrow \mathcal{A}_1(U)$ then $\mathbb{P}[i = i_0] = \frac{\text{wt}(i_0)}{w}$. On the other hand, if $(i, j) \leftarrow \mathcal{A}_2(U)$ then i is uniform over $\{i \in [m] : \text{wt}(i) > 0\}$. In order to understand $\mathcal{F}(U)$, we need to study the probability of a value (i_0, j_0) being output in a given iteration of the loop in $\mathcal{F}$. This is $\mathbb{P}[i = i_0 \wedge \text{Filter}_k(U, i, j) = 1]$ where $(i, j) \leftarrow \mathcal{A}_1(U)$. Since $\text{Filter}_k(U, i, j)$ ignores j , we can consider the two events as independent and rewrite the probability as:

$$p_{\mathcal{F}}(i_0) = \mathbb{P}[i = i_0 \wedge \text{Filter}_k(U, i_0, \perp) = 1] = \mathbb{P}[i = i_0] \cdot \mathbb{P}[\text{Filter}_k(U, i_0, \perp) = 1].$$

We know that $\mathbb{P}[i = i_0] = \frac{\text{wt}(i_0)}{w}$, so now we will to study $\mathbb{P}[\text{Filter}_k(U, i_0, \perp) = 1]$. Assume $\text{wt}(i_0) > 0$. For a fixed c_0 , we have $\mathbb{P}[\text{Filter}_k(U, i_0, \perp) = 1 | c = c_0] = \mathbb{P}[r \leq$

$c_0] = \min\left(1, \frac{c_0}{k\ell}\right)$. We also know that $\mathbb{P}[c = c_0] = \rho(i_0)(1 - \rho(i_0))^{c_0-1}$, meaning $\mathbb{P}[c = c_0 + c_1] = (1 - \rho(i_0))^{c_1} \mathbb{P}[c = c_0]$. So, by the law of total probability

$$\begin{aligned}
\mathbb{P}[\text{Filter}_k(U, i_0, \perp) = 1] &= \sum_{c_0=1}^{\infty} \min\left(1, \frac{c_0}{k\ell}\right) \cdot \mathbb{P}[c = c_0] \\
&= \sum_{c_0=1}^{k\ell} \frac{c_0}{k\ell} \mathbb{P}[c = c_0] + \sum_{c_0=k\ell+1}^{\infty} \mathbb{P}[c = c_0] \\
&= \sum_{c_0=1}^{\infty} \frac{c_0}{k\ell} \mathbb{P}[c = c_0] + \sum_{c_0=k\ell+1}^{\infty} \left(1 - \frac{c_0}{k\ell}\right) \mathbb{P}[c = c_0] \\
&= \sum_{c_0=1}^{\infty} \frac{c_0}{k\ell} \mathbb{P}[c = c_0] - \sum_{c_0=k\ell+1}^{\infty} \frac{c_0 - k\ell}{k\ell} \mathbb{P}[c = c_0] \\
&= \sum_{c_0=1}^{\infty} \frac{c_0}{k\ell} \mathbb{P}[c = c_0] - \sum_{c_0=1}^{\infty} \frac{c_0}{k\ell} \mathbb{P}[c = c_0 + k\ell] \\
&= \sum_{c_0=1}^{\infty} \frac{c_0}{k\ell} \mathbb{P}[c = c_0] - \sum_{c_0=1}^{\infty} \frac{c_0}{k\ell} (1 - (1 - \rho(i_0))^{k\ell}) \mathbb{P}[c = c_0] \\
&= \frac{1}{k\ell} (1 - (1 - \rho(i_0))^{k\ell}) \sum_{c_0=1}^{\infty} c_0 \mathbb{P}[c = c_0] \\
&= \frac{1}{k\ell} (1 - (1 - \rho(i_0))^{k\ell}) \mathbb{E}[c] \\
&= \frac{1}{k\ell} (1 - (1 - \rho(i_0))^{k\ell}) \frac{1}{\rho(i_0)}.
\end{aligned}$$

Thus $p_{\mathcal{F}}(i_0) = \frac{n}{wk\ell} (1 - (1 - \rho(i_0))^{k\ell})$. If we can control $(1 - \rho(i_0))^{k\ell}$, then we can make $p_{\mathcal{F}}(i_0) \approx p_*(i_0)$, where $p_*(i_0) = \frac{1}{wk\ell}$. Define distributions $\mathcal{D}_{\mathcal{F}}$ and $\mathcal{D}_*$ on $[m] \cup \{\perp\}$ which assign probabilities $p_{\mathcal{F}}(i_0)$ and $p_*(i_0)$ as defined above to each $i_0 \in [m]$ with $\text{wt}(i_0) > 0$, set $p_{\mathcal{F}}(i_0) = p_*(i_0) = 0$ for each $i_0 \in [m]$ with $\text{wt}(i_0) = 0$, and assign the remaining probability to $p_{\mathcal{F}}(\perp)$ and $p_*(\perp)$. We will show in Lemma B.2 below that $(1 - \rho(i_0))^{k\ell} \leq \text{negl}(\lambda)$ for all i_0 with $\text{wt}(i_0) > 0$. It follows that

$$\begin{aligned}
\sum_{i_0=1}^m |p_{\mathcal{F}}(i_0) - p_*(i_0)| &= \sum_{\text{wt}(i_0) > 0} |p_{\mathcal{F}}(i_0) - p_*(i_0)| \\
&= \sum_{\text{wt}(i_0) > 0} \left| \frac{n}{wk\ell} (1 - (1 - \rho(i_0))^{k\ell}) - \frac{n}{wk\ell} \right| \\
&= \frac{n}{wk\ell} \sum_{\text{wt}(i_0) > 0} (1 - \rho(i_0))^{k\ell} \\
&\leq \frac{mn}{wk\ell} \max_{\text{wt}(i_0) > 0} (1 - \rho(i_0))^{k\ell} \\
&\leq \frac{mn}{wk\ell} \text{negl}(\lambda).
\end{aligned}$$

The expected weight w (taken over the random choice of U) is $\mathbb{E}[w] = \frac{mn}{\ell}$, and by Hoeffding's inequality

$$\mathbb{P}\left[w < \frac{mn}{2\ell}\right] = \mathbb{P}\left[\frac{mn}{\ell} - w > \frac{mn}{2\ell}\right] \leq \exp\left(-\frac{mn}{\ell^2}\right) \leq \text{negl}(\lambda)$$

We may therefore assume $w \geq \frac{mn}{2\ell}$, so $\sum_{i_0=1}^m |p_{\mathcal{F}}(i_0) - p_*(i_0)| \leq \frac{2}{k} \text{negl}(\lambda)$. By a similar argument,

$$\begin{aligned} |p_{\mathcal{F}}(\perp) - p_*(\perp)| &= \left| \left(1 - \sum_{i_0=1}^m p_{\mathcal{F}}(i_0)\right) - \left(1 - \sum_{i_0=1}^m p_*(i_0)\right) \right| \\ &= \left| \sum_{i_0=1}^m (p_{\mathcal{F}}(i_0) - p_*(i_0)) \right| \\ &\leq \text{negl}(\lambda). \end{aligned}$$

We conclude that $\mathcal{D}_{\mathcal{F}}$ and $\mathcal{D}_*$ have statistical distance $\Delta(\mathcal{D}_{\mathcal{F}}, \mathcal{D}_*) \leq \text{negl}(\lambda)$. Therefore we may assume each iteration of the loop in $\mathcal{F}$ outputs i_0 with probability $\frac{n}{wk\ell}$ for each i_0 with $\text{wt}(i_0) > 0$. This shows that the distribution $\{i : (i, j) \leftarrow \mathcal{F}(U)\}$ is statistically indistinguishable from the uniform distribution on $\{i \in [m] : \text{wt}(i) > 0\}$, which, as stated above, is the distribution of $\{i : (i, j) \leftarrow \mathcal{A}_1(U)\}$. This proves property 3 in the definition of Filtering (Definition 4.8). Property 1 is immediately true, and we can now show property 2 (that $p_{\text{accept}} = \mathbb{P}[\text{Filter}_k(U, i, j) = 1 | (i, j) \leftarrow \mathcal{A}_1(U)]$ is non-negligible) as well:

$$p_{\text{accept}} = \sum_{i_0=1}^m p_{\mathcal{F}}(i_0) \approx \sum_{\text{wt}(i_0) > 0} \frac{n}{wk\ell} \geq \sum_{\text{wt}(i_0) > 0} \frac{\text{wt}(i_0)}{wk\ell} = \frac{1}{k\ell}.$$

Finally, we show that the error $(1 - \rho(i))^{k\ell}$ can be made negligibly small.

Lemma B.2. *If $k \geq 2\lambda\ell(\lambda + \ln(m))$, then with overwhelming probability over the choice of U we have $(1 - \rho(i))^{k\ell} \leq \text{negl}(\lambda)$ for all $i \in [m]$ with $\text{wt}(i) > 0$.*

Proof. First, suppose $n \leq 2\ell^2(\lambda + \ln(m))$. So $k \geq 2\lambda\ell(\lambda + \ln(m)) \geq \frac{n\lambda}{\ell}$. Since $\text{wt}(i) \geq 1$, we have

$$(1 - \rho(i))^{k\ell} = (1 - \rho(i))^{n\lambda} = \left(1 - \frac{\text{wt}(i)}{n}\right)^{n\lambda} \leq \left(1 - \frac{1}{n}\right)^{n\lambda} \leq e^{-\lambda}.$$

Now suppose $n > 2\ell^2(\lambda + \ln(m))$. By Hoeffding's inequality, for any particular i we have

$$\begin{aligned}\mathbb{P}\left[\rho(i) < \frac{1}{2\ell}\right] &= \mathbb{P}\left[\frac{\text{wt}(i)}{n} < \frac{1}{2\ell}\right] \\ &= \mathbb{P}\left[\frac{1}{\ell} - \frac{\text{wt}(i)}{n} > \frac{1}{2\ell}\right] \\ &\leq \exp\left(-\frac{n}{2\ell^2}\right) \\ &\leq \exp\left(-\frac{2\ell^2(\lambda + \ln(m))}{2\ell^2}\right) \\ &\leq \frac{1}{m}e^{-\lambda}.\end{aligned}$$

Thus, by the union bound, $\mathbb{P}[\exists i \in [m] : \rho(i) < \frac{1}{2\ell}] \leq e^{-\lambda}$. We may therefore assume that $\rho(i) > \frac{1}{2\ell}$ for all $i \in [m]$. Since $k \geq 2\lambda\ell(\lambda + \ln(m)) \geq 2\lambda$, we have

$$(1 - \rho(i))^{k\ell} \leq (1 - \rho(i))^{2\lambda\ell} \leq \left(1 - \frac{1}{2\ell}\right)^{2\lambda\ell} \leq e^{-\lambda}.$$

C Deferred Security Proofs

Here we present the deferred proofs of Lemmas 4.1, 4.2, and 4.3. Recall that AS refers to Base ORCAS, depicted in 9.

Lemma C.1 (Canonical Signing). *AS has strongly canonical signing.*

Proof. Let $(Y, y) \in \mathcal{R}$ and $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$, and fix a message m . A signature $\sigma \leftarrow \text{Sign}(\text{sk}, m)$ is uniquely determined by the tuple of commitments $(\text{comm}_1, \dots, \text{comm}_t)$, which is distributed uniformly over $C_1 \sqcup \dots \sqcup C_t$ (the disjoint union means we count multiplicity), where $C_r = \{(\text{comm}_1, \dots, \text{comm}_t) \in X^t : \text{chal}_r = 0\}$. In fact $(\text{comm}_1, \dots, \text{comm}_t)$ is sampled uniformly from X^t . It may, however, be rejected depending on the value of $\text{chal} = H(m || \text{comm}_1 || \dots || \text{comm}_t)$: a commitment is used if and only if $\text{chal}_r = 0$, where $r \leftarrow_{\$} [t]$. Thus it is clear that the final commitment is sampled uniformly from $C_1 \sqcup \dots \sqcup C_t$.

A pre-signature $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$ is uniquely determined by its own tuple of commitments $(\text{comm}'_1, \dots, \text{comm}'_t)$, along with its choice of r . Since two pre-signatures with the same commitments but different choices of r will both adapt to the same signature, the output of $\text{CanonicalSign}_{(Y, y)}$ is uniquely determined by $(\text{comm}'_1, \dots, \text{comm}'_t)$. Therefore, it suffices to show that $(\text{comm}'_1, \dots, \text{comm}'_t)$ is uniform over $C_1 \sqcup \dots \sqcup C_t$, which follows from the previous reasoning done for $(\text{comm}_1, \dots, \text{comm}_t)$. $\square$

Lemma C.2 (Pre-Signature Correctness). *AS is pre-signature correct.*

Proof. For pre-signature correctness we will show three properties:

1. For a given keypair (sk, pk) , message m , tuple $(Y, y) \in \mathcal{R}$, and pre-signature $\hat{\sigma} \leftarrow \text{PreSign}(\text{sk}, m, Y)$, we have $\text{PreVerify}(\text{pk}, m, Y, \hat{\sigma}) = 1$.
2. After adapting the pre-signature $\hat{\sigma}$ into the signature $\sigma := \text{Adapt}(\hat{\sigma}, y)$, we have $\text{Verify}(\text{pk}, m, \sigma) = 1$.
3. After extracting $y' := \text{Ext}(\sigma, \hat{\sigma}, Y)$, we have $(Y, y') \in \mathcal{R}$.

Let $\lambda \in \mathbb{N}, m \in \{0, 1\}^*$ and $(Y, y) \in \mathcal{R}$. Given $\text{sk}_i \in G$ for $i = 1, \dots, S-1$, then we compute $x_i = \text{sk}_i * x_0$, for all $i = 1, \dots, S-1$. Consider keys $\text{sk} = (\text{sk}_1, \dots, \text{sk}_{S-1})$ and $\text{pk} = (x_1, \dots, x_{S-1})$, where each $x_i = \text{sk}_i * x_0$.

To show *property 1*, let $\hat{\sigma} = (r, \text{chal}, \widehat{\text{resp}}_1, \dots, \widehat{\text{resp}}_t) \leftarrow \text{PreSign}(\text{sk}, m, Y)$. Notice that asking the couple (Y, y) is in $\mathcal{R}$ implies in particular that Y is in $\mathcal{L}$. Then the first PreVerify check is satisfied. Moreover, for each $i \neq r$, the pre-signer computes $\text{comm}_i = g_i * x_0$ and $\widehat{\text{resp}}_i = g_i \cdot \text{sk}_{\text{chal}_i}^{-1}$, while the pre-verifier computes $\text{comm}'_i = \widehat{\text{resp}}_i * x_{\text{chal}_i}$. So, $\text{comm}'_i = (g_i \cdot \text{sk}_{\text{chal}_i}^{-1}) * x_{\text{chal}_i} = g_i * x_0 = \text{comm}_i$. For $i = r$, the pre-signer computes $\text{comm}'_r = g_r * Y$ and $\widehat{\text{resp}}_r = g_r$, while the pre-verifier computes $\text{comm}'_r = \widehat{\text{resp}}_r * Y = g_r * Y = \text{comm}_r$. Having shown that $\text{comm}'_i = \text{comm}_i$ for each $i \in [t]$, we know

$$\text{chal}' = H(m || \text{comm}'_1 || \dots || \text{comm}'_t) = H(m || \text{comm}_1 || \dots || \text{comm}_t) = \text{chal}.$$

The honesty of the pre-signature $\hat{\sigma}$ guarantees us that $\text{chal}_r = 0$. Therefore $\text{PreVerify}(\text{pk}, m, Y, \hat{\sigma}) = 1$.

To show *property 2*, let $\sigma = (\text{chal}, \text{resp}_1, \dots, \text{resp}_t) = \text{Adapt}(\hat{\sigma}, y)$. For each $i \neq r$, the adaptor computes $\text{resp}_i = \widehat{\text{resp}}_i$ while the verifier computes $\text{comm}'_i = \text{resp}_i * x_{\text{chal}_i}$. Once again, we have $\text{comm}'_i = \text{comm}_i$. For $i = r$, the adaptor computes $\text{resp}_r = \widehat{\text{resp}}_r \cdot y$ while the verifier computes $\text{comm}_r = \text{resp}_r * x_{\text{chal}_r}$. Since $\text{chal}_r = 0$, we have $\text{comm}'_r = (\widehat{\text{resp}}_r \cdot y) * x_0 = \widehat{\text{resp}}_r * Y = \text{comm}_r$. Again,

$$\text{chal}' = H(m || \text{comm}'_1 || \dots || \text{comm}'_t) = H(m || \text{comm}_1 || \dots || \text{comm}_t) = \text{chal}.$$

This is the verifier's only check, so $\text{Verify}(\text{pk}, m, \sigma) = 1$.

To show *property 3*, let $y' = \text{Ext}(\sigma, \hat{\sigma}, Y)$. Since σ and $\hat{\sigma}$ have the same challenge chal , the extractor will not return $\perp$; instead, it will return

$$y' = \widehat{\text{resp}}_r^{-1} \cdot \text{resp}_r = \widehat{\text{resp}}_r^{-1} \cdot (\widehat{\text{resp}}_r \cdot y) = y.$$

Therefore $(Y, y') = (Y, y) \in \mathcal{R}$. □

Lemma C.3 (Pre-Signature Adaptability). *AS is pre-signature adaptable.*

Proof. Let $(Y, y) \in \mathcal{R}, m \in \{0, 1\}^*$, and $\text{pk} = (x_1, \dots, x_{S-1})$. Consider a pre-signature $\hat{\sigma} = (r, \text{chal}, \text{resp}_1, \dots, \text{resp}_t)$ such that $\text{PreVerify}(\text{pk}, m, Y, \hat{\sigma}) = 1$, and let $\sigma = (\text{chal}, \text{resp}_1, \dots, \text{resp}_t) \leftarrow \text{Adapt}(\hat{\sigma}, y)$. For all $i \neq r$, since $\text{resp}_i = \widehat{\text{resp}}_i$ we have the same comm'_i in both PreVerify and Verify . For $i = r$, we know from pre-verification that $\text{chal}_r = 0$ and thus $\text{resp}_r * x_{\text{chal}_r} = (\widehat{\text{resp}}_r \cdot y) * x_0 = \widehat{\text{resp}}_r * Y$. So comm'_r is also the same in both PreVerify and Verify , meaning so is chal' and thus $\text{Verify}(\text{pk}, m, \sigma) = 1$. □